ENHANCING MUSIC EMOTION RECOGNITION (MER) FOR EMOTION-BASED MUSIC RECOMMENDATION SYSTEM USING GENERATIVE ADVERSARIAL NETWORKS (GANs)

Ganegama Vithanage Dona Imasha Kethmie Perera
(2426583)

M.Sc. in Data Science

October 2025

Supervisor: Dr. Stuart Cunningham

School of Computing and Engineering Sciences
Faculty of Science, Business and Enterprise

Abstract

Music is a very emotional element as it is a strong medium through which human beings express and feel. This correlation of music with emotion is what is the focus of this study to understand and replicate. The proposed dissertation introduces a system to improve Music Emotion Recognition (MER) and emotion-based music recommendation based on Generative Adversarial Networks (GANs), in the Pop Piano genre.

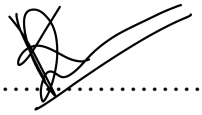
This research starts by deriving rich musical attributes of MIDI files with *prettyMIDI* library, which include pitch, rhythm, harmony and dynamics. The classification of emotions is then done through a threshold-based model as a result of research-inspired feature boundaries of six emotions namely *happy*, *sad*, *excited*, *hopeful*, *fearful*, and *tense*. Since the diversity of emotion was small in EMOPIA, GAN-based data augmentation was used to balance the underrepresented emotion classes synthetic to enhance consistency of the dataset and model generalisation.

The effectiveness of the proposed system was tested both quantitatively (accuracy measures, GAN loss curves, and similarity scores) and qualitatively (human validation of the system by the study of 30 participants). Measured on the basis of Fleiss Kappa, there was a little-to-fair inter-rater agreement between participants, which once again confirms the subjectivity of the emotional perception in music. With a similarity tracking favouring similar songs, the recommendation module was found to be accurate in intra-emotion accuracy with 100% accuracy.

The contribution that this work can offer to the fields of "emotional" computing and Music Information Retrieval (MIR) lies in the fact that data augmentation and emotion modelling have been incorporated into a single pipeline. It also identifies the opportunity of GANs to improve emotion recognition tasks and prepares the future study of making emotion-feature thresholds standardised, emotion-based recommendation models validated by larger human studies, and cross-modal affective understanding of music technology.

Disclaimer

This work is original and has not been previously submitted in support of any other course or qualification.



…….

Ganegama Vtihanage Dona Imasha Kethmie Perera

09.10.2025

Acknowledgements

I would like to express my sincere gratitude to everyone who supported me throughout the completion of this dissertation.

First and foremost, my sincere thanks go to my supervisor, Dr Stuart Cunningham, for his guidance, encouragement, and expertise. His thoughtful feedback and insightful suggestions helped refine this research and gave it both academic and practical depth.

I am also grateful to the University of Chester and the academic staff of the MSc Data Science programme for providing the knowledge, tools, and learning environment that made this work possible. The combination of technical and research-based modules allowed me to approach this project with both analytical and creative understanding.

I would like to extend heartfelt thanks to the thirty participants who took part in my survey. Your contributions provided the foundation for the human evaluation aspect of this study and gave this research its real-world significance.

To my family, especially my parents, thank you for your unconditional love, sacrifices, and encouragement. Your belief in me gave me the strength to pursue my studies and persevere through every challenge.

To my partner, Manilka, thank you for your endless patience, understanding, and encouragement. Your support during the long days of coding and writing made this journey lighter and more meaningful.

Finally, to my friends and classmates, thank you for your advice, motivation, and companionship throughout this course.

This dissertation marks both an academic achievement and a personal milestone, and I am deeply grateful to everyone who helped make it possible.

Table of Contents

1 Introduction 8
1.1 Background 8
1.2 Problem Statement 8
1.3 Hypothesis 9
1.4 Research Aim and Objectives 9
1.5 Significance of the Study 10
1.6 Structure of the Dissertation 10
2 Literature Review 11
2.1 History of Music Information Retrieval (MIR) 11
2.2 Brief History of MER (Music Emotion Recognition) 12
2.3 Models of Emotion in MER 13
2.4 Limitations of current MER approaches 13
2.5 The use of GANs (Generative Adversarial Networks) in MER 14
2.6 MER in Recommendation Systems 14
2.7 Datasets 14
2.8 Summary 15
3 Methodology 16
3.1 Research Design 16
3.2 Dataset 16
3.2.1 Feature Extraction 17
3.3 Generative Adversarial Networks (GANs) for Augmentation 17
3.3.1 Training Procedure: 18
3.4 Emotion Classification 18
3.4.1 Criteria of classification: 18
3.5 Optimisation and Loss Functions. 19
3.6 Evaluation Metrics 20
3.6.1 Classification Performance: 20
3.6.2 GAN Training Metrics: 20
3.6.3 Recommendation Quality: 20
3.7 Human Subject Testing 20
3.7.1 Implementation of Fleiss Kappa 21
3.8 Ethical Considerations 24
3.9 Summary 24
4 Implementation 25

4.1 Technologies and Libraries 25
4.2 System Architecture Implementation 25
4.3 Feature Extraction Module Implementation 27
4.3.1 Pitch and Velocity Features 28
4.3.2 Tempo and Temporal Features 28
4.3.3 Harmony and Rhythm Features 29
4.3.4 Articulation and Dynamic Feature 29
4.4 Emotion Classification Module Implementation 30
4.4.1 Rule-based Classification Logic 31
4.4.2 Dataset-wide Classification and Distribution 31
4.5 Data Augmentation Module Implementation 31
4.5.1 Network Architecture 32
4.5.2 Model Training 33
4.5.3 Data Preparation and Scaling 34
4.5.4 Synthetic Sample Generation 34
4.5.5 Dataset balancing 35
4.6 Recommendation System Module Implementation 35
4.6.1 Model training 36
4.6.2 Finding Similar tracks 36
4.7 Exploratory Data Analysis Module Implementation 37
4.7.1 Summary and Inspection of Datasets. 37
4.7.2 Dataset Comparison: Original vs Balanced 38
4.7.3 GAN Training Visualisation 38
4.8 Summary 39
5 Testing and Results 40
5.1 Exploratory Data Analysis (EDA) 40
5.1.1 Emotion Distribution 40
5.1.2 GAN Training Analysis 41
5.2 Human Evaluation and Inter-rater Agreement 42
5.2.1 Fleiss' Kappa Calculation 43
5.3 System Outputs and Emotion Classification Results 45
5.4 Recommendation System Evaluation 46
5.5 Summary 49
6 Discussion and Conclusion 51
6.1 Discussion and Key Findings 51
6.1.1 Emotional feature thresholds and lack of research correlation 51
6.1.2 Variability in human emotional perception 51

6.1.3 Effectiveness of GAN-based data augmentation 52
6.1.4 Comparison between EMOPIA classification and System classification 52
6.1.5 Verification of emotion-based recommendations produced by the system 52
6.2 Conclusion 53
Appendices 61
Appendix A – Code 61
Main.py 61
Eda_analysis.py 64
emotion_classifier.py 70
feature_extractor.py 72
recommendation_system.py 76
simple_gan.py 78
Fleiss-kappa.py 82
Appendix B – List of Figures 83
Fleiss Kappa Agreement System Results 83
Recommendation System Results per track 84
Appendix C – Ethical Approval 92
Appendix D – Survey Results 93

1 Introduction

1.1 Background

Music is considered to be a strong tool for expressing and arousing feelings (P. N. Juslin & Västfjäll, 2008). This emotional aspect of music has gained increased significance in the digital era for music recommendation systems that seek to give users tailored as well as interesting listening experiences (Priya et al., 2025). Collaborative filtering or acoustic similarity are often used in more conventional recommender systems, which do not always succeed in capturing the affective properties of music that make people satisfied (Cheng et al., 2020). Music Emotion Recognition (MER) attempts to overcome this drawback, by modelling the emotional content of music, using computational techniques, which enables systems to more closely recommend songs that match the mood or intended affective response of a listener.

MER is going through continuous challenges despite the improvement of machine learning and deep learning tools. The first and foremost is the lack and unbalanced nature of labelled emotional datasets, as it restricts the capacity of models to generalise over music genres and emotional classes (Eerola & Anderson, 2025). The recent advances in the field of Generative Adversarial Networks (GANs) have demonstrated that this kind of synthetic data can help balance datasets even in the fields that are close to speech emotion recognition and EEG-based affective recognition (Bao et al., 2021; Latif et al., 2020), where synthetic data has been proved to be effective at enhancing classification performance. Nevertheless, the use of GANs on MER is poorly studied, especially on pop piano music, which is a genre that can be analysed by emotion since it has a fixed arrangement and can be expressed through tone. This report explores the ability of GAN-based augmentation to benefit MER and enhance the performance of emotion-based recommendation systems.

1.2 Problem Statement

The main challenge of creating accurate and generalisable MER systems is the lack of high-quality labelled data. Current datasets are usually unbalanced in their representation of the categories of emotions, where certain ones (e.g., sadness or calmness) are overrepresented and others (e.g., fear or excitement) are underrepresented. Such an imbalance causes biased models which are poorly performing on underrepresented classes, and at the end, this reduces the performance of the downstream applications, including recommendations (Eerola & Anderson, 2025; Kang & Herremans, 2025). Although GANs have shown excellent results in synthetic data

generation in other areas, very little has been done with GANs applied to the task of supplementing music emotion datasets. Such lack of approaches limits the development of the MER and limits its possible implementation in the affect-aware recommender systems.

1.3 Hypothesis

The hypotheses that guide this study are:

- H1: GAN-based feature augmentation enhances the performance of music emotion classifier relative to that of classifier trained on the original EMOPIA dataset (Hung et al., 2021).
- H2: MIDI-based symbolic features are more likely to generate higher classification accuracy compared to either of the feature sets individually.
- H3: The musical features considered have direct connection to human perception of emotion.
- H4: The music that the system recommends based on emotions is consistent with those that are perceived by the listeners, which shows the relevance of the strategy in practice.

1.4 Research Aim and Objectives

The purpose of the research is to examine how the GAN-based augmentation can be used to enhance the MER of pop piano music and how it can influence the emotion-driven recommendation systems.

To do this, the research is organized with respect to the following objectives:

1. Find out what MER applications are available and what their strengths and weaknesses are.
2. Evaluate and analyse the use of GAN in emotion recognition tasks in similar fields.
3. Train a GAN model to augment the features of pop piano music.
4. Assess the success of GAN enhanced data to classify MER.
5. Adopt a system of recommendations based on emotions and evaluate how it is compatible with human perception.

The main research question is: Is it possible to use GANs to improve emotional music data and enhance MER models of pop piano music?

1.5 Significance of the Study

Although GANs have been largely used in speech and EEG-based emotion recognition (Bao et al., 2021; Latif et al., 2020), their use in MER, especially in data augmentation, is minimal. This study produces novel findings on the topic of fine-grained modelling of emotion in music by concentrating on the EMOPIA dataset (Hung et al., 2021) -and generalizing its four-quadrant valence-arousal labels to six more granular ones (*happy*, *excited*, *tense*, *fearful*, *sad* and *hopeful*). In addition, by combining GAN-augmented MER with a recommendation system proves the usefulness of the method in personalised and affect-conscious music recommendation. The results can guide the future research in the field of music informatics, designing a recommender system, and emotion modelling.

1.6 Structure of the Dissertation

The rest of this dissertation is structured in the following way:

- Chapter 2: Literature Review: The chapter reviews the history of MIR and MER, summarizes available models and methods and discusses the application of GANs and their limited use in music, as well as research gaps, especially data augmentation and recommendation.
- Chapter 3: Methodology describes the research design, datasets, feature extraction methods (symbolic and audio), GAN architecture, and evaluation metrics that were used.
- Chapter 4: Implementation describes the system design and develop process, and feature engineering, training a GAN, integrating classifiers, and building a recommendation pipeline.
- Chapter 5: Testing and Results introduces the experimental findings including the classification performance with and without GAN augmentation, statistical analysis and recommendation evaluation and comparison with human perception.
- Chapter 6: Discussion and Conclusion summarising the results, assesses the extent to which the hypotheses were confirmed, takes a retrospective view of the work and suggests future research directions.

2 Literature Review

Music Information Retrieval (MIR) has provided the basis for analysing and organising music using computational means, resulting in genre classification, melody search, and similarity analysis. Out of these capabilities, the Music Emotion Recognition (MER) has developed, which entails forecasting the emotion performed or perceived in music. MER has also transformed rule-based methods to machine learning and deep learning strategies and is backed by various psychological models of emotion and is used in applications like music recommendation.

In spite of this, MER has issues such as subjectivity, small and unbalanced data, and integration with generative methods. Although Generative Adversarial Networks (GANs) have demonstrated promising results in the field, they are little used in MER. Having access to datasets, such as EMOPIA, where audio is paired with symbolic information, it is possible to investigate how GAN-based augmentation may enhance the process of classification and recommendation. This review describes the evolution of MIR and MER, their models, applications and the gaps that exist in the research that prompted this study.

2.1 History of Music Information Retrieval (MIR)

MIR is an integrative research that includes processing, analysing, organising, and accessing music information with the use of computational methods (Dixon et al., 2018; Downie, 2003; *Editorial: Introducing the Transactions of the International Society for Music Information Retrieval | Transactions of the International Society for Music Information Retrieval*, n.d.). This was introduced due to the growing need to manage and access expanding digital music collections, beginning in the 1990s. Early MIR solution for this was the query-by-humming system, which retrieves corresponding features from a database depending on the melody that the user hums (Ghias et al., 1995). With further advancement, audio fingerprinting was introduced, which allowed robust recognition of music clips even under background noise and distortion – this is widely known to be the underlying technology used in Shazam (Wang, 2003). This key research demonstrated that music could be systematically represented through features such as pitch sequences, spectral signatures and rhythmic cues, which opened the possibility of identifying patterns and analysing music features further. This included moving towards the expressive side of music, which is recognising emotion through music.

2.2 Brief History of MER (Music Emotion Recognition)

MER is an interdisciplinary field that focuses on the recognition of emotions conveyed or perceived by music (Y.-H. Yang & Chen, 2012). Research on MER initially focused primarily on music psychology. To illustrate this, Hevner (1936) formulated musical parameters, where tempo, mode, and harmony convey certain emotions: suggesting that faster tempos and major modes are perceived as happy or exciting, and slow tempos and minor modes as evoking sadness. These results were essential for interpreting the relationships between music and emotions that they imply, but these studies were too narrow, as well as inapplicable to various musical genres and cultural settings.

With the compilation of annotated datasets of musical extracts and associated emotional labels, the next stage of MER began, characterised by the adoption of data-driven methods. These datasets were the first step towards supervised machine learning models, which could map musical features, including mel-frequency cepstral coefficients (MFCC), chroma, tempo, note density, and spectral centroid, into emotion categories. Support vector machines, k-nearest neighbours, and random forests classification algorithms were also popular at this time, and regression models were used to estimate continuous values of valence (pleasantness) and arousal (activation) (Y.-H. Yang & Chen, 2012). This change was a definite step toward abandoning prescriptive rules in favour of an empirical modelling of statistical relationships between music features and perceived emotion.

As deep learning has gained popularity, techniques of MER have evolved. Music spectrograms of musical signals have been analysed using convolutional neural networks (CNNs), which encode musical timbral and musical rhythmic information (Choi et al., 2016). Recurrent neural networks (RNNs) and long short-term memory (LSTM) have enabled researchers to investigate tempo dependencies in music, and attention mechanisms (Chang et al., 2024) allow systems to focus on important musical events during the process of evaluating emotions (Li et al., 2022; Qiao et al., 2024). Although the methods are superior to the traditional methods, they require significantly bigger annotated datasets, which points to a persistent problem in the area (Eck & Schmidhuber, n.d.).

Eerola & Anderson (2025) reported on an extensive meta-analysis of MER research carried out from 2014 to 2024 and including 34 studies and 204 models. They discovered that regression models, which predict valence and arousal, had r = 0.67 and r = 0.81 mean correlations, respectively, and categorical classification models had mean accuracies of about 0.87 with the help of the Matthews correlation coefficient (Chicco & Jurman, 2020). Interestingly, the meta-

analysis revealed that simpler machine learning models tend to be as effective as more complex neural networks in regression tasks; however, deep learning models tend to be more effective than classical models in categorical classification. Their review also highlighted limitations, such as inconsistency in labelling practices, a lack of cross-cultural generalisation, an imbalance in datasets, and a lack of transparency in reporting methodology.

2.3 Models of Emotion in MER

One of the theoretical bases of MER is the choice of an emotion model. The Russell circumplex model of affect is one of the most popular theories, and it conceptualises emotion on two continuous axes: valence (pleasant/unpleasant) and arousal (activated/calm) (Russell, 1980). This model is especially applicable to music since musical attributes like tempo, pitch, and dynamics are closely associated with these dimensions: faster tempos and greater loudness will tend to positively affect arousal, whereas major mode harmonies will tend to affect valence positively.

Other models have been employed too. The adjective circle of Hevner (1936) described emotions in terms of descriptive words (e.g., joyful, solemn), whereas Thayer (1989) energy-tension model introduced energy and stress dimensions equivalent to those of arousal and valence. The Geneva Emotional Music Scales (GEMS) (Zentner et al., 2008) provided a music-specific taxonomy comprising such categories as wonder, nostalgia, and tenderness, although the scheme is too elaborate to be used in computational research.

In practice, Russell's circumplex model continues to be the most widely used because of its simplicity, cultural robustness, and compatibility with datasets like EMOPIA (Hung et al., 2021).

2.4 Limitations of current MER approaches

Despite these advancements, there remains significant challenges with traditional MER approaches. Firstly, emotional subjectivity suggests that the perceived emotions may vary significantly among listeners, across cultures, and in different situations. Secondly, label inconsistency between datasets complicates the comparison of studies: some of them are discrete, and others continuous valence-arousal ratings. Thirdly, there is a lack of data and its imbalance restricts the generalisability of the model (e.g., one emotion may be overrepresented, whereas another may not be). Lastly, deep learning tools tend to be more restrictive regarding their ability to generalise since they require bigger annotated datasets than are currently available (Eerola & Anderson, 2025). These shortcomings highlight the importance of new methods like data augmentation and generative modelling.

2.5 The use of GANs (Generative Adversarial Networks) in MER

In other similar applications, GANs have been shown to have strong potential in the aspect of generating synthetic data (e.g., speech emotion recognition, where GANs generate synthetic spectrogram features to balance training data (Latif et al., 2020), and EEG emotion recognition, where adversarial networks generate synthetic signals to enhance the performance of classifiers (Bao et al., 2021)). GANs on the context of music, have however, been studied in music generation, and not recognition. Examples for these are: MuseGAN to generate multi-track symbolically, (Dong et al., 2017), and MidiNet to generate melodies conditional on chord sequences (L.-C. Yang et al., n.d.). More recently, Neves et al., (2022) used GANs for the sentiment-conditioned generation of symbols. However, little research had been done on MER-based GANs in the context of data augmentation, which is a significant gap in the literature.

2.6 MER in Recommendation Systems

Emotion-based music recommendation systems (EMRS) aim at improving customer experience by choosing songs that align with, or control the mood of the user. Initial systems were based on manual mood tags that were labour-intensive and fixed. Less rigid methods visualize tracks in valence-arousal space and suggest those that are represented in the immediate vicinity (Tran et al., 2023). Hybrids integrate features of content with collaborative filtering systems, and occasionally with contextual or physiological input like heart rate or activity (Schedl et al., 2018). Practical examples of commercial systems such as mood-based playlists (Spotify) reveal that there are still difficulties in matching the system-predicted and user-perceived emotion, despite academic research (Siles et al., 2019). Assessment of EMRS is a significant weakness: whereas the accuracy of recommendations can be quantitatively assessed, the affective relevance usually needs to be evaluated through user studies. This complicates comparisons that are made across systems.

2.7 Datasets

The MER research has been advantaged by having a variety of datasets, such as:

- DEAM (Dataset to Emotion Analysis with Music) (Soleymani et al., n.d.): annotated with valence and arousal values of the Western popular music.
- Million Song Dataset (Bertin-Mahieux et al., 2011): includes metadata and audio features and has no fine-grained emotional annotations.
- EMOPIA (Hung et al., 2021): a multimodal data set of piano pop performances that were annotated based on the four quadrants of Russell.

EMOPIA is one that has an impact on this study. To begin with, it concentrates on piano pop music, which is comparatively uniform in the choice of instruments and results in fewer confounded timbral elements as well as better identification of structural characteristics. Second, EMOPIA has aligned symbolic (MIDI) and audio data, so it can be used to integrate structural and expressive capabilities. Last but not least, its annotations are directly correlated to the circumplex model of Russell (Russell, 1980) which provides theoretical consistency. These properties precondition the fact that EMOPIA is especially appropriate in the study of GAN-based augmentation and recommendation.

2.8 Summary

The literature review helps to identify the research gap: GANs have been successfully used to perform data augmentation with speech and EEG emotion recognition, but the problem of music emotion recognition remains unexplained, particularly as a solution to the problem of class imbalance. Besides that, there is an even smaller literature that has incorporated GAN-based augmentation into end-to-end recommendation systems, in such a way that augmentation data enhances not only the quality of classification but also the quality of emotion-sensitive recommendations.

The proposed study aims to fill this gap by using GAN-based augmentation on the EMOPIA dataset, clustering its quadrants into 6 categories of emotions (happy, excited, tense, fearful, sad, hopeful), and assessing the effect of such changes on classification accuracy and recommendation performance. This approach helps advance MER methodologies and provides new insights into the practicability of integrating data augmentation and recommendation in emotion-aware systems.

3 Methodology

This chapter elaborates on the research methodology used for developing this project. The methodology provides the conceptual and procedural foundation for how considered research hypotheses are tested. This ensures that the outcomes of this research can be recreated, scientifically accepted, and in line to improve MER accuracy and recommendation quality. The research uses a qualitative, experimental research design that integrates both computational modelling and human validation. Thus, through this chapter, the reader will gain an understanding of dataset preparation, feature configuration, GAN-based data augmentation, classifier training, music recommendation and statistical evaluation with human participants.

3.1 Research Design

The proposed study is based on a computational-experimental design to examine how GANs can enhance MER through the addition of new data, and in the case of Pop Piano genre. The process has four primary steps, namely: (i) extraction of MIDI features of the EMOPIA dataset, (ii) threshold-based classification of emotions, (iii) data augmentation and classification with the help of a GAN-based system, and (iv) evaluation with quantitative metrics and a human-centred viability.

The system is designed using best practices in MER research, which is usually a combination of signal processing, machine learning, and human perception studies. The reasons why GANs are adopted for this system are based on their success in producing realistic synthetic data in other fields of emotion-related tasks, like speech emotion recognition and EEG emotion recognition (Bao et al., 2021; Latif et al., 2020), where GANs have been utilised to solve data scarcity and class-imbalanced data.

3.2 Dataset

The dataset of the study is based on the EMOPIA dataset (Hung et al., 2021) which is a multimodal corpus of 1,087 pieces in piano genre annotated with four-quadrant (4Q) labels of valence and arousal. In order to accommodate finer emotion recognition, this study deepens the 4Q scheme of EMOPIA into 6 emotion clusters: Happy, Excited, Tense, Fearful, Sad and Hopeful.

The EMOPIA dataset was chosen as it is highly detailed in terms of covering emotional music and its annotation approach is well-documented. The data is based on piano pop music and aligns with the research aim, which is to create a recommendation system for similar music based on

the pop piano genre. The MIDI format offers musical features that are important in emotion classification and the methodology of annotation used in the dataset also gives good quality emotion label that could be used in validation.

3.2.1 Feature Extraction

The system derives 15 different musical features of MIDI files with the help of the *PrettyMIDI* (Raffel et al., 2014). library, and this system classifies these features into eight categories:

- Pitch Features: Minimum, maximum, mean, and standard deviation of pitch values (indicates the melody and how emotion is expressed via pitch height and contour)
- Velocity Features: Minimum, maximum, mean, and standard deviation of note velocities (indicates the intensity and energy of performance).
- Tempo Features: Minimum, maximum, mean, and standard deviation of tempo values (required to identify emotional states).
- Harmony Features: Chord complexities and structure measures.
- Rhythm Features: Density of notes and complexity of rhythm.
- Articulation Characteristics: Characteristics of note constructions, the legato and staccato style of playing.
- Temporal Features: Total duration of music clip and temporal distribution.
- Dynamic Features: Change of loudness and intensity during the performance.

All these features form a wholesome illustration of musical features that can represent both symbolic and perceptual features of musical expression and form the basis of emotion classification and recommendation generation.

3.3 Generative Adversarial Networks (GANs) for Augmentation

This section explains the central element of the methodology, which is the application of GAN architecture to solve the issue of imbalance in the dataset by synthesising samples. GAN implementation is based on the established best practices in generative modelling and is simple and interpretable to researchers (Goodfellow et al., 2014). The *SimpleGAN* is made up of two neural networks:

- Generator (G): Feedforward network which produces synthetic feature vectors given an input which is random noise (christianversloot, n.d.).
- Discriminator (D): This is a smaller network in comparison that is trained to differentiate real and synthetic feature vectors.

The adversarial training is based on the usual minimax game formulation, with the Generator trying to generate realistic feature vectors to fool the Discriminator and the Discriminator trying to learn to recognise synthetic samples correctly.

3.3.1 Training Procedure:

GAN training is done in few steps:

- Data Preparation: The feature vectors are normalised with the help of *StandardScaler*.
- Balanced Training: GAN models of the different categories of emotions to ensure that category-specific properties arise in generated samples.
- Progressive Training: The training activity involves loss functional monitoring ensure stable training dynamics.

The implementation solves the issue of dataset imbalance through the creation of synthetic samples whereby imbalanced categories of emotions are only created, the augmented dataset preserves the integrity of the original data and balances the categories of emotions.

3.4 Emotion Classification

The classification system of emotions uses a threshold-based system of classifying musical pieces into six categories of emotions (Happy, Sad, Excited, Hopeful, Tense, and Fearfu) using musical features. The thresholds of the classification system are based on the existing music theory and the empirical studies of music-emotion correlations.

In contrast to most of the current Music Emotion Recognition (MER) systems, which operate based on the supervised model of learning (e.g., Hung et al., 2021; Qiao et al., 2024), this research follows a rule-based classification system built on the threshold values of musical features. This was to avoid the unavailability of emotion labels at the ground-truth in the EMOPIA dataset, and yet enable meaningful mapping of musical characteristics and expression of emotion.

The logic of classifications was adopted in the *EmotionClassifier*, which applies thresholds on features extracted through MIDI, i.e. pitch, velocity, tempo, note-overlap ratio. A number of feature ranges characterise each of the emotion categories (Refer Table 3.1).

3.4.1 Criteria of classification:

The system will use a mechanism of scoring that will assess the extent of fit between the extracted features and emotion threshold. The algorithm will compute the percentage similarity of each category of emotions and select the one that has a high percentage of similarity, provided

it has a threshold of 50 percent similarity. In this way, it guarantees classifications, and it is also transparent in the decision-making process.

Table 3.1 – Emotion Classification Thresholds

Emotion	pitch_min	pitch_max	velocity_min	velocity_max	tempo_min	tempo_max	note_overlap_ratio_min	note_overlap_ratio_max	Reference
Happy	60	76	80	112	120	140	0	0.4	Eerola & Vuoskoski (2013) – high valence = higher pitch & faster tempo; Juslin & Laukka (2004) – happy = fast tempo, major mode;
Sad	48	60	40	70	60	80	0.8	1	Eerola & Vuoskoski (2013); Juslin & Laukka (2004); Yang & Chen (2012) – low arousal = slower tempo, lower pitch;
Excited	65	84	100	127	110	180	0.25	0.65	Yang & Chen (2012) – high arousal = faster tempo, louder dynamics; Juslin & Laukka (2004) – excited = loud, fast, staccato;
Hopeful	60	76	70	100	100	130	0.6	0.8	**No direct data found, hence, assumptions made** – inferred from "moderate tempo, moderate valence" in Schubert (2004); modeled as blend of "happy" and "tense" musical properties
Fearful	45	60	50	90	60	100	0	0.3	Krumhansl (1997) – fearful music uses dissonant chords, minor modes; Lartillot & Toiviainen (2007) – tonal tension increment with pitch / dynamics;
Tense	50	65	90	127	100	130	0	0.3	Eerola & Vuoskoski (2013); Poli et al (2007) – tension = high tempo, large dynamics;

Since there is no existing standard that dictates specific threshold values of musical features, the ranges used in this research were developed by the researcher by synthesis of results of several earlier studies (reference details mentioned in Table 3.1) and modified experimentally.

3.5 Optimisation and Loss Functions.

The system utilises different optimization techniques and loss functions, which are specific to every part of the pipeline:

- GAN Training: Losses Generator and Discriminator networks are trained by using Binary Cross-Entropy (BCE). BCE compares the mean similarity to the true label (real=1, fake=0) and prompts the Generator to generate realistic feature vectors as it trains the Discriminator to recognise well synthetic samples.
- Feature Scaling: StandardScaler is used to standardise all features to make sure no musical characteristic contributes more to the training process and the calculation of the similarity.
- Training Monitoring: The training process includes the overall loss-function monitoring, convergence measuring, and training stability.

3.6 Evaluation Metrics

The system employs multiple evaluation metrics to assess performance across different aspects of the pipeline:

3.6.1 Classification Performance:

- Accuracy: Proportion of correct emotion classifications across all categories. Following equation demonstrates how this is calculated:

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Number\ of\ Samples} \times 100$$

3.6.2 GAN Training Metrics:

- Generator Loss: Measures the Generator's ability to produce realistic samples.
- Discriminator Loss: Measures the Discriminator's ability to distinguish real from synthetic samples.
- Training Stability: loss curve monitoring to ensure stable training dynamics.

3.6.3 Recommendation Quality:

- Similarity Accuracy: Percentage of recommendations that match the query's emotion category. Following equation demonstrates how this is calculated:

$$Similarity\ Accuracy = \frac{Number\ of\ Emotion\ Matched\ Recommendations}{Total\ Number\ of\ Recommendations} \times 100$$

3.7 Human Subject Testing

A listening study was performed with 30 human participants to investigate the relationship between the outputs of the system match the human perception. This was conducted on an online setting where the participants joined a Google Meet meeting and the host (the author) played 25 Pop Piano music tracks by EMOPIA. The participants were then asked to classify these tracks into one of the six target feelings (namely, *Happy*, *Sad*, *Tense*, *Hopeful*, *Excited* and *Fearful*). These responses were obtained through Google Forms.

To preserve consistency among participants should meet the following criteria:

- Over 18 of age
- Currently not experiencing hearing difficulties
- Understands spoken and written English

Further, information on how the participants is listening to the tracks are collected to ensure that they hear the tracks properly.

Fleiss Kappa (Fleiss, 1971) was used as a statistical measure of inter-rater agreement, with 0.60 as a reasonable threshold to regard a number as high agreement between two raters (Landis & Koch, 1977).

3.7.1 Implementation of Fleiss Kappa

3.7.1.1 Step 1: Compute Proportions per Category (P_{ij})

For each track i, proportion of ratings that fall into each emotion category j is calculated. This represents how the 30 participants votes are distributed across the six emotions for that track.

$$P_{ij} = \frac{n_{ij}}{n}$$

where:

- n_{ij} - number of raters who selected category $j$for track i
- n - total number of raters per track (30 in this research)

3.7.1.2 Step 2: Agreement per Track (P_i)

For each track, this calculates how much agreement there was among raters. If all raters agreed, $P_i = 1$; if ratings are evenly split, P_i approaches 0

$$P_i = \frac{1}{n(n-1)} \sum_{j=1}^{k} n_{ij}(n_{ij} - 1)$$

Where:

- k= number of emotion categories (6 in this research)

3.7.1.3 Step 3: Mean Observed Agreement ($\bar{P}$)

This calculates the average agreement across all tracks to get the overall observed agreement.

$$\bar{P} = \frac{1}{N} \sum_{i=1}^{N} P_i$$

where:

- N= number of tracks (25 in this research)

3.7.1.4 Step 4: Expected Agreement by Chance ($\bar{P}_e$)

This computes how much agreement is to be expected just by chance if raters randomly picked emotions.

$$P_j = \frac{1}{Nn}\sum_{i=1}^{N} n_{ij}$$

$$\bar{P}_e = \frac{1}{Nn}\sum_{j=1}^{k} {p_j}^2$$

where p_j shows the overall proportion of all ratings assigned to category j.

3.7.1.5 Step 5: Fleiss' Kappa (κ)

Finally, this calculates the Fleiss' Kappa coefficient, which adjusts observed agreement ($\bar{P}$) for chance agreement ($\bar{P}_e$)

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

Interpretation (Landis & Koch, 1977):

- 0.01–0.20 = Slight agreement
- 0.21–0.40 = Fair agreement
- 0.41–0.60 = Moderate agreement
- 0.61–0.80 = Substantial agreement
- 0.81–1.00 = Almost perfect agreement

Following is the code-level implementation of these five-step approach:

```
import pandas as pd
import numpy as np
from statsmodels.stats.inter_rater import fleiss_kappa

df = pd.read_csv("survey-results.csv")

emotion_cols = ["Happy", "Excited", "Tense", "Fearful", "Sad", "Hopeful"]

# p_ij - proportion of raters per emotion per track
n = 30
pij = df[emotion_cols] / n
print("\nSTEP 1: Proportions (p_ij):")
print(pij.round(3))

# P_i - agreement per track
pi = (df[emotion_cols] * (df[emotion_cols] - 1)).sum(axis=1) / (n * (n - 1))
print("\nSTEP 2: Agreement per track (P_i):")
print(pi.round(3))

# Mean observed agreement (P')
P_bar = pi.mean()
print(f"\nSTEP 3: Mean observed agreement (P'): {P_bar:.3f}")

# p_j - overall proportion per emotion and expected agreement (P'_e)
pj = df[emotion_cols].sum(axis=0) / (len(df) * n)
P_e_bar = (pj ** 2).sum()
print("\nSTEP 4: Category proportions (p_j):")
print(pj.round(3))
print(f"Expected agreement by chance (P̄_e): {P_e_bar:.3f}")

# Fleiss' Kappa
kappa = (P_bar - P_e_bar) / (1 - P_e_bar)
print(f"\nSTEP 5: Fleiss' Kappa (κ): {kappa:.3f}")

# Interpretation (Landis & Koch, 1977)
if kappa < 0.00:
    interpretation = "Poor agreement"
elif kappa < 0.21:
    interpretation = "Slight agreement"
elif kappa < 0.41:
    interpretation = "Fair agreement"
elif kappa < 0.61:
    interpretation = "Moderate agreement"
elif kappa < 0.81:
    interpretation = "Substantial agreement"
```

3.8 Ethical Considerations

The University of Chester Research Ethics Committee granted ethical approval for the involvement of human subjects. The participants were told the purpose of the study, their rights (including the right to withdraw) as well as promised anonymity. They were provided with informed consent before being involved. No personal details were withheld and the data were stored safely in line with the GDPR and institutional policies. Please refer to the Approval Letter in Appendix C.

3.9 Summary

The framework of music emotion analysis and recommendation system development provided in this methodology chapter has provided the answer to the main difficulties in music information retrieval by implementing the innovative methods toward data augmentation, emotion classification, and the development of recommendations. The research approach combines a variety of disciplines, such as music theory, machine learning, and human perception research to develop a sound and scientifically rigorous research framework. The research design has several critical contributions to the field of music emotion recognition, in the form of the four-quadrant system of emotion model of the EMOPIA dataset extension to a six-category system, the use of a SimpleGAN system of data augmentation and the creation of a threshold-based system of emotion classification offering interpretable and transparent decision-making processes.

4 Implementation

This chapter provides a detailed technical description of how the GAN-based MER and recommendation system is implemented, further, how it links the conceptual design established in the earlier chapter with the final code implementation. The chapter illustrates the conversion of the theoretical model into working software, including an explanation of design choices, software structure and implementation plans. The software is applied in a modular manner, and thus it is maintainable, scalable and reproducible and is consistent with the research objectives.

4.1 Technologies and Libraries

Python was used to develop the system, which was selected due to its flexibility, readability, and the availability of scientific and machine learning libraries.

The basis of data processing and numerical calculation was developed in the form of *Pandas* and *NumPy*, which allows manipulating extracted musical features and provides the effective processing of the MIDI files EMOPIA dataset.

To process the MIDI data, the *prettyMIDI* library was heavily employed to read the MIDI files and identify the symbolic musical characteristics like the pitch, velocity, tempo, the length of notes and the timing data. This library allowed converting raw MIDI signals to quantifiable measurements which could be further interpreted to obtain emotional traits.

The GAN was implemented using the *TensorFlow* library, including the *Keras*. *Keras* is a high-level interface to build and train the networks of generators and discriminators that balanced the dataset by type of emotion.

The *scikit-learn* was applied in various parts of the system. It also facilitated the process of data scaling through *StandardScaler* that all the extracted features were normalised prior to model training and also to compute the cosine similarity.

To visualise and conduct EDA, *Matplotlib* and *Seaborn* were used.

These libraries combined offered a capable and stable technological backbone to complete a full MER and recommendation pipeline, such as MIDI functionality extraction and classifications, GAN-based augmentation and emotion-sensitive music suggestions.

4.2 System Architecture Implementation

The system architecture is executed as a set of Python classes, which wrap the functionality of all the modules. The orchestration consists of the *main.py* script, which organises the execution of all the components in the right sequence. The modular design will make each component

testable and verifiable independently, and the interfaces among the modules will make the systems easier to maintain and improve.

The system is pipeline-based, with data passing through each module:

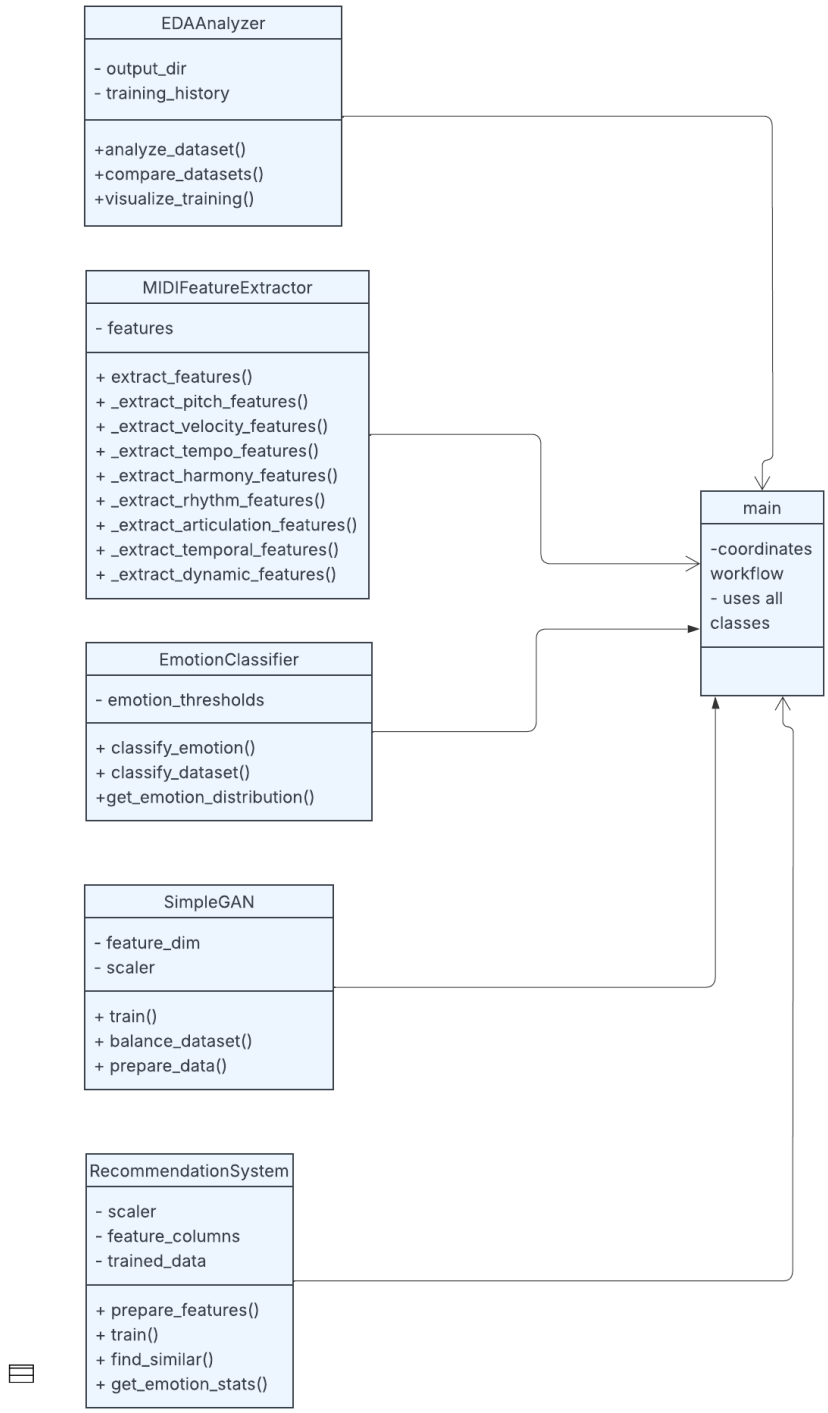


Figure 4.1 – Class Diagram of the system

The feature extraction module is responsible for converting MIDI files into structured feature vectors. The emotion classification module relies on the rule-based thresholds to categorise the

features derived. The data augmentation module is used to equalise the dataset by synthesising fake examples, and the recommendation module is used to learn familiarity patterns so as to make purposeful recommendations

The modular structure followed by the project abides by the principles of object-oriented design (P.E. et al., 2023), and methods are grouped by feature type to ensure code maintainability.

4.3 Feature Extraction Module Implementation

The feature extraction component, named as *MIDIFeatureExtractor*, is a class that captures all the functionality of MIDI files processing and musical features extraction. This class will be extracting and structuring a set of 15 core features that includes: pitch, tempo, velocity, harmony, rhythm, articulation, temporal and dynamic features:

```
class MIDIFeatureExtractor:
    def __init__(self):
        self.features = []

    def extract_features(self, midi_path):
        """Extract all features from a MIDI file"""
        try:
            midi_data = pretty_midi.PrettyMIDI(midi_path)
            features = {}

            # Basic features
            features['file_path'] = midi_path
            features['duration'] = midi_data.get_end_time()

            # Pitch features
            features.update(self._extract_pitch_features(midi_data))

            # Velocity features
            features.update(self._extract_velocity_features(midi_data))

            # Tempo features
            features.update(self._extract_tempo_features(midi_data))

            # Harmony features
            features.update(self._extract_harmony_features(midi_data))

            # Rhythm features
            features.update(self._extract_rhythm_features(midi_data))

            # Articulation features
            features.update(self._extract_articulation_features(midi_data))
```

```
            # Temporal features
            features.update(self._extract_temporal_features(midi_data))

            # Dynamic features
            features.update(self._extract_dynamic_features(midi_data))

            return features

        except Exception as e:
            print(f"Error processing {midi_path}: {e}")
            return None
```

pretty_midi.PrettyMIDI() is responsible in loading MIDI track's even data (such as notes, instruments, timing, etc.). The *extract_feature()* function encapsulates all the lower-level feature extraction functions.

4.3.1 Pitch and Velocity Features

The module initially calculates the pitch-related features (*pitch_min, pitch_max, pitch_mean, pitch_std*) through *_extract_pitch_features()* and velocity-related features (*velocity_min, velocity_max, velocity_mean, velocity_std*) through *_extract_velocity_features()*

```
            'pitch_min': min(all_notes),
            'pitch_max': max(all_notes),
            'pitch_mean': np.mean(all_notes),
            'pitch_std': np.std(all_notes)
```

These values are obtained by iterating through all notes of each instrument in the MIDI file. Pitch values describe the tonal range and the variation whereas, the velocity values describe the intensity and the dynamic activity of the notes.

```
            'velocity_min': min(all_velocities),
            'velocity_max': max(all_velocities),
            'velocity_mean': np.mean(all_velocities),
            'velocity_std': np.std(all_velocities)
```

4.3.2 Tempo and Temporal Features

Tempo-related features such as *tempo_mean* and *tempo_std* are extracted to measure the average tempo as well as its variation throughout a piece.

```
            'tempo_mean': np.mean(tempo_changes[1]),
            'tempo_std': np.std(tempo_changes[1])
```

Further, *tempo_variation* and duration are also extracted through *_extract_temporal_features()* function. These will measure the fluctuations in tempo changes and the overall playback length of the file.

```
            'tempo_variation': tempo_variation,
            'duration': midi_data.get_end_time()
```

4.3.3 Harmony and Rhythm Features

Chord_changes and *harmony_complexity* are extracted by the *_extract_harmony_features()* function. This will be calculated by counting overlapping notes.

To perform rhythm analysis, *_extract_rhythm_features()* calculates mean and standard deviation of note duration (*rhythm_mean, rhythm_std*) and then obtains *syncopation* value, which represents the rhythmic irregularity.

```
        'harmony_complexity': chord_changes / max(1, len(midi_data.instruments))
        'syncopation': syncopation
```

4.3.4 Articulation and Dynamic Feature

Articulation shows how notes are played relative to each other. *note_overlap_ratio* feature indicates the number of times a note overlaps over time whereas, *total_notes* feature counts the number of notes within the track.

```
            'note_overlap_ratio': overlap_ratio,
            'total_notes': total_notes
```

Finally, the *_extract_dynamic_features()* function compares the change in velocity among the track in order to calculate *dynamic_range* (maximum difference between loudest and softest notes) and *dynamic_contrast* (total variation of loudness).

```
            'dynamic_range': dynamic_range,
            'dynamic_contrast': dynamic_contrast
```

4.4 Emotion Classification Module Implementation

The Emotion Classification Module takes the extracted numerical features in each of the MIDI files and puts them into one of six categories of emotion, Happy, Sad, Hopeful, Excited, Tense and Fearful. This classification is implemented in the *EmotionClassifier* class which uses the feature-threshold rules based on previous Music Emotion Recognition (MER) studies.

The module is not based on machine-learning training, rather, it is rule-based and deterministic to assess the best fitting emotion profile to a given feature set of a track. Through this strategy it is possible to observe how single musical characteristics (e.g. pitch, tempo, or note overlap) influence perception of emotion:

```
class EmotionClassifier:
    def __init__(self):
        self.emotion_thresholds = {   …   }
```

The six categories of emotions are set up at the initial level where a dictionary of feature thresholds characterises each emotion category.

The ranges include *pitch_mean*, *velocity_mean*, *tempo_mean*, *note_overlap_ratio* for each emotion (threshold ranges are mentioned under 3.4.1. Criteria of classification topic). The values involved were based on many researches, which have analysed the correlation between musical parameters and emotional aspects like arousal and valence (Eerola & Vuoskoski, 2013; P. N. Juslin & Laukka, 2004; P. Juslin & Sloboda, 2013; Krumhansl, 1997; Lartillot & Toiviainen, n.d.; Lin et al., 2013; Poli et al., 2007; Russell, 1980).

As an example, high-pitch and high-tempo tones have always been associated with high-arousal positive-valence emotions such as happiness and excitement, and low-pitch, slow-tempo, and high-ratio legato correlate with low-arousal negative-valence emotions such as sadness and fearfulness (Eerola & Vuoskoski, 2013; Fernández-Sotos et al., 2016).

4.4.1 Rule-based Classification Logic

To all the emotions, the algorithm scores a representation of the percentage of criteria met:

```
if thresholds['pitch_min'] <= features['pitch_mean']<=thresholds['pitch_max']:
   score += 1
```

Even though the Feature Extraction Module calculates 15 parameters, this system depends on four major ones (*pitch_mean*, *velocity_mean*, *tempo_mean*, and *note_overlap_ratio*).

The reason why this design was selected is the fact that there is limited research that has defined quantitative threshold values of all the emotional features in music. Therefore, the existing thresholds were added based on the **rationalised approximations** of numerous sources (e.g. such as, P. Juslin & Sloboda (2013)). Future research would be able to narrow down these thresholds to large scale annotated datasets or by incorporating machine-learning frameworks that could automatically learn feature-emotion mappings.

Once all the conditions are verified, emotion score within 0 and 1 is obtained. The feeling with the most points is attributed to the song in case the match is more than 0.5 (50 percent of thresholds met):

```
    if emotion_scores:
        best_emotion = max(emotion_scores, key=emotion_scores.get)
        if emotion_scores[best_emotion] >= 0.5:
            return best_emotion
```

4.4.2 Dataset-wide Classification and Distribution

The resulting labelled data can then be analysed with *get_emotion_distribution(),* which shows the frequency count of the predicted emotions. This gives a preview of the emotional balance of the dataset and is used subsequently as the input of GAN-based augmentation.

4.5 Data Augmentation Module Implementation

Data Augmentation Module was created to deal with imbalance between categories of emotions in the EMOPIA dataset after categorising the dataset further into six categories. GAN was developed using *TensorFlow* and *Keras* to artificially create new synthetic data to tackle this issue.

```
class SimpleGAN:
    def __init__(self, feature_dim, noise_dim=100):
        self.feature_dim = feature_dim
        self.noise_dim = noise_dim
        self.scaler = StandardScaler()
```

As shown above, the module was initialised with two parameters:

- *Feature_dim*: number of numerical features extracted per MIDI file
- *Noise_dim*: vector length used to generate synthetic feature patterns

StandardScaler is also initialised as it will be used to normalise the feature distributions before GAN training. This provides numerical stability.

4.5.1 Network Architecture

GAN consists of two neural networks: a generator and a discriminator. Both these networks are implemented as *Keras* sequential models.

1. Generator (G):

The generator is responsible for transforming random noise into synthetic feature vectors that imitate real music-feature data. This network contains two hidden layers, first layer with *ReLU* activations and the output later with *tanh* activations, which will be matching feature dimensions, constraining values between -1 and 1:

```
def _build_generator(self):
        """Build simple generator"""
        model = keras.Sequential([
            keras.layers.Dense(256, activation='relu', input_dim=self.noise_dim),
            keras.layers.Dense(512, activation='relu'),
            keras.layers.Dense(self.feature_dim, activation='tanh')
        ])
```

This design using *tanh* provides smooth value distributions which can be used as symbolic-music features, such as pitch or velocity.

2. Discriminator (D):

The responsibility of the discriminator is to learn to differentiate between real and synthetic feature vectors. This network too contains two layers, first with *ReLU* activations and second with *sigmoid* for binary classification (where, real = 1, fake = 0):

```
    def _build_discriminator(self):
        """Build simple discriminator"""
        model = keras.Sequential([
            keras.layers.Dense(512, activation='relu',
input_dim=self.feature_dim),
            keras.layers.Dense(256, activation='relu'),
            keras.layers.Dense(1, activation='sigmoid')
        ])
```

During adversarial training, both models are trained at the same time where the generator generate synthetic feature vectors whereas discriminator tries to differentiate between the real and the synthetic data.

4.5.2 Model Training

These models are trained with the Adam optimiser and the learning rate of 0.0002 and *Beta_1* of 0.5, which are commonly used in GAN studies to stabilise convergence. The discriminator is training with binary cross-entropy loss and accuracy as a metric whereas, only binary cross-entropy loss is used by the combined model (generator and discriminator):

```
        # discriminator
        self.discriminator.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5),
            loss='binary_crossentropy',
            metrics=['accuracy']
        )

        # combined model
        self.combined.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5),
            loss='binary_crossentropy'
        )
```

The training process (in *train()* function) is repeated 100 epochs in a batch size of 32. In each emotion class, real samples are chosen randomly in the dataset, and the generator generates fake samples.

This is done by training the discriminator on a mixed set of real and synthetic samples, and training the generator to deceive the discriminator. The progress of training (discriminator loss D-loss and generator loss G-loss) can also be monitored through the *eda-analyzer*, which logs performance variables to be used in the visualisation of performance later on:

```
 if epoch % 20 == 0:
    d_loss_val = d_loss[0] if isinstance(d_loss, (list, np.ndarray)) else d_loss
    g_loss_val = g_loss[0] if isinstance(g_loss, (list, np.ndarray)) else g_loss
    print(f"Epoch {epoch}, D Loss: {d_loss_val:.4f}, G Loss: {g_loss_val:.4f}")
```

As a rule, when the D-loss is stabilised at 1.0 with 15-20 epochs, this means the discriminator is moderately certain about the ability to differentiate between real and fake inputs and the generator is continuing to improve creating synthetic data.

4.5.3 Data Preparation and Scaling

The *prepare_data()* function is filters the dataset based on emotion label, and only the columns of the features (numerical features) are selected. They are brought to zero mean and unit variance to fit within the range of activation of the generator.

```
        # numeric features only
        numeric_features =
emotion_data.select_dtypes(include=[np.number]).drop(['emotion'], axis=1,
errors='ignore')

        # Scale features
        scaled_features = self.scaler.fit_transform(numeric_features)
```

Scaling is needed since such features like pitch, velocity, tempo are used on different numerical scales (e.g., 0-127 velocity, 60-180 BPM tempo). In the absence of standardisation, the large-scale features would take over training and distort the distribution of the output of the generator.

4.5.4 Synthetic Sample Generation

New synthetic samples are generated after training through feeding random noise vectors to the generator.

The generated features are inverse-transformed back to their original scale using the fitted *StandardScaler*, then added to a DataFrame with the relevant emotion label:

```
        # Generate noise
        noise = np.random.normal(0, 1, (n_samples, self.noise_dim))

        # Generate features
        generated_features = self.generator.predict(noise, verbose=0)

        # Inverse transform to original scale
        generated_features = self.scaler.inverse_transform(generated_features)
```

The process enables the system to generate new and plausible combinations of features that reflect the statistical patterns of real tracks, especially of those emotions that are underrepresented.

4.5.5 Dataset balancing

The method *balance_dataset()* evaluates the existing emotion distribution and is used to automatically create other samples into categories less than a target (a default of 200 samples per emotion)

```
        for emotion, count in emotion_counts.items():
            if emotion == 'unknown':
                continue

            if count < target_samples_per_emotion:
                needed = target_samples_per_emotion - count
                print(f"Generating {needed} samples for {emotion}")

                # Get existing data for this emotion
                emotion_data = df[df['emotion'] == emotion]
                data, columns = self.prepare_data(df, emotion)

                if data is not None:
                    # Generate new samples
                    generated = self.generate_samples(needed, emotion_data,
columns)
                    balanced_df = pd.concat([balanced_df, generated],
ignore_index=True)
```

This makes sure that whatever categories of emotion are to be represented, are equally represented prior to the next stages of classification and evaluation. Consequently, the balanced dataset will reduce the bias of the models in the majority classes (e.g., Happy or Sad) and make the recognition of emotions and recommendation results more robust.

4.6 Recommendation System Module Implementation

The last step of the implementation pipe is the Recommendation System Module. It is meant to propose emotionally similar tracks out of the EMOPIA data set in terms of the similarity of emotion category of a particular input track and its musical features. The system employs cosine similarity to measure closeness between tracks to suggest similar feature-based tracks:

```
        # Calculate similarities
        similarities = []
        for idx, row in same_emotion_data.iterrows():
            if row['file_path'] == query_file_path:
                continue  # Skip the query file itself

            # Skip if file_path is NaN or empty
            if pd.isna(row['file_path']) or row['file_path'] == '':
                continue

            features = row['scaled_features'].reshape(1, -1)
            similarity = cosine_similarity(query_features, features)[0][0]
            similarities.append((row['file_path'], similarity))

        # Sort by similarity and return top N
        similarities.sort(key=lambda x: x[1], reverse=True)
```

Cosine Similarity was selected to this as it compares the similarity between all musical features than being conformed into one category, further, as the track is classified according to feature thresholds, in theory, cosine similarity should produce the same category.

4.6.1 Model training

The *train()* method normalises the data and the metadata and contains both the scaled feature vectors and the metadata of each track. It basically creates the feature table that will be employed by the system to find similar objects.

```
        self.trained_data = df.copy()
        self.trained_data['scaled_features'] = list(features)
```

Each track is then symbolised by its high dimensional feature vectors, which captures its symbolic musical characteristics, standardised and open to comparison.

4.6.2 Finding Similar tracks

The most important part of the recommendation process is *find_similar()* function. When a user enters some query track (MIDI file path), the method calculates the feature vector of this track and finds other tracks of the same emotion type.

The decision to limit comparison to a same emotion category and category guarantees consistency of the emotion in the results of the recommendations. GAN-generated data is excluded when making similarity comparisons to ensure that real files are used for recommendation:

```
# Get all files with same emotion (ONLY from original dataset, not GAN-generated)
        # Filter out GAN-generated samples by checking if file_path is not NaN
and exists
        same_emotion_data = self.trained_data[
            (self.trained_data['emotion'] == query_emotion) &
            (self.trained_data['file_path'].notna()) &
            (self.trained_data['file_path'] != '') &
            (self.trained_data['file_path'].str.contains('.mid', na=False))
        ]
```

This is followed by the use of cosine similarity between the query feature vector and that of all the other tracks:

```
            features = row['scaled_features'].reshape(1, -1)
            similarity = cosine_similarity(query_features, features)[0][0]
            similarities.append((row['file_path'], similarity))
```

The measure used to determine the angular distance of vectors, giving figures ranging between -1 (exactly opposite) to 1 (exactly the same).

Here, the greater the cosine similarity (the closer the value is to 1.0), the higher the similarity in features and hence the probability of emotional and structural similarity of the tracks. After sorting the results, system returns 5 similar tracks to the user.

4.7 Exploratory Data Analysis Module Implementation

To gain diagnostic information about the dataset structure, the impact of GAN-based augmentation, and the system behaviour in general, the Exploratory Data Analysis (EDA) Module was used.

This component has an analytical role and a validation role, i.e. studying the data at the beginning and after balancing and keeping track of the training stability of the GAN over the epochs.

The implementation is done with the help of the *EDAAnalyzer* class, which is calculated with the help of *Pandas*, *NumPy*, *Matplotlib*, and *Seaborn*

4.7.1 Summary and Inspection of Datasets.

Functions under this module includes *analyze_dataset()* and *_basic_info()* which pull out information about general dataset properties including total samples, number of features, missing values and memory consumption. These simple measures are necessary to ensure data integrity prior to any form of model training:

```
        print(f"Total samples: {len(df)}")
        print(f"Total features: {len(df.columns)}")
        print(f"Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f}
MB")
        print(f"Missing values: {df.isnull().sum().sum()}")
        print(f"Duplicate rows: {df.duplicated().sum()}")
```

This is done to make sure that the MIDI feature dataset extracted is complete and contains no duplicate entries, and it is not so large that it can exceed memory limits.

These are also used to ensure that clean and standardised input data is fed to the GAN and recommendation modules.

4.7.2 Dataset Comparison: Original vs Balanced

One of the main analytical elements of this module is the *compare_datasets()* method. It contrasts the distribution of emotions prior and after to GAN augmentation, and measures the extent to which the shift in the balance between emotion categories was achieved.

```
        # Emotion distribution comparison
        original_dist = original_df['emotion'].value_counts()
        balanced_dist = balanced_df['emotion'].value_counts()

        comparison_df = pd.DataFrame({
            'Original': original_dist,
            'Balanced': balanced_dist
        }).fillna(0)
```

The difference in emotional representation between datasets is plotted using bar plots which are stored in *dataset_comparison.png* in the results directory.

4.7.3 GAN Training Visualisation

Dynamic visualisation of the training performance of the GAN is also offered by the module by *record_training_step()* and *visualize_training()*. These functions monitor and visualise the values of discriminator loss (D-loss) and generator loss (G-loss) per epoch, allowing convergence and stability to be analysed:

```
axes[0, 0].plot(self.training_history['epochs'],
self.training_history['discriminator_loss'], 'b-', label='Discriminator Loss',
linewidth=2)
axes[0, 0].plot(self.training_history['epochs'],
self.training_history['generator_loss'], 'r-', label='Generator Loss',
linewidth=2)
```

The result of visualisation (*gan_training_analysis.png*) contains six diagnostic plots, including:

- Loss curves over time
- D/G loss ratio
- Loss change rate
- Smoothed loss convergence
- Stability metrics (mean and standard deviation)
- Phase-wise loss trends (early, mid, late training)

A combination of these plots gives an idea of whether GAN training stabilised to some equilibrium. The final loss values with their variability and general classification of the quality of training (stable, moderate, or unstable) are also printed in the console.

4.8 Summary

This chapter has presented an in-depth technical description of the implementation of the MER and recommendation system. The implementation process shows how the theoretical framework presented in the Methodology chapter is applied to actual software components, which may be implemented and tested in practice.

Its application is done with best software engineering practices, such as modular design, robust error handling, and documentation. The code architecture also guarantees the ability to test and validate every component separately, and the explicit boundaries between modules make maintaining and improving the system easier.

The choice of technology stack is a balance between functionality, performance and development ease. Python, along with proven machine learning libraries, means that the system can be easily replicated and built upon by other researchers. The application proves that the theoretical framework is successfully implemented such that it can satisfy the research goals and be reproducible.

5 Testing and Results

This chapter discusses the evaluation, testing and analysis of the research. The purpose of this chapter is to assess the performance of the developed system and compare the results with human emotional perception. Evaluation process is conducted in two perspectives:

1. System-based testing – Examines the performance of the classifier before and after GAN-based data augmentation.
2. Human-based testing – Discusses the Survey conducted with 30 participants, which categorises the emotional content of 25 pop piano tracks taken from the EMOPIA dataset.

The results discussed throughout this chapter are outputs of the final system (as printed in the terminal and saved files through *main.py* execution), combined with survey responses collected through a Google Form. EDA and other visualisations are used to understand both data distribution and the model's learning patterns. Together, these results provide an in-depth understanding on the performance of the model and how closely it replicates human emotion perception in music.

5.1 Exploratory Data Analysis (EDA)

EDA was conducted through the eda_analysis.py module, which visually and statistically illustrates the structure of the dataset before and after data augmentation.

5.1.1 Emotion Distribution

The Emotion Distribution (Figure 5.1) illustrates that the EMOPIA dataset initially was imbalanced after being categorised based on music feature thresholds.

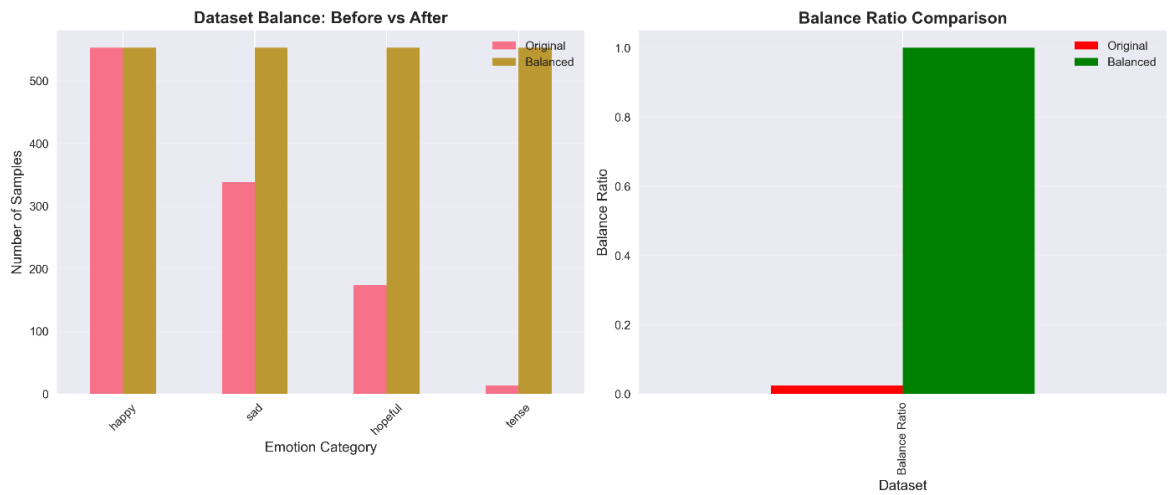


Figure 5.1 – Emotion Distribution before and after GAN

As per the music feature thresholds provided through research (refer topic 3.4 Emotion Classification), EMOPIA dataset does **not** accompany all six emotional categories considered in this research. The original EMOPIA dataset when applied through *EmotionClassifier*, categorises EMOPIA MIDI data into four emotional categories, namely, *Happy*, *Sad*, *Hopeful* and *Tense*. GAN is then performed on the four categories to ensure a balanced dataset.

5.1.2 GAN Training Analysis

The GAN was then trained for 50 epochs with a batch size of 32. The training process alternated between generator and discriminator, recording loss values at every 20th epoch. Decreasing and increasing loss fluctuations of both networks over time demonstrates the stability of GAN training:

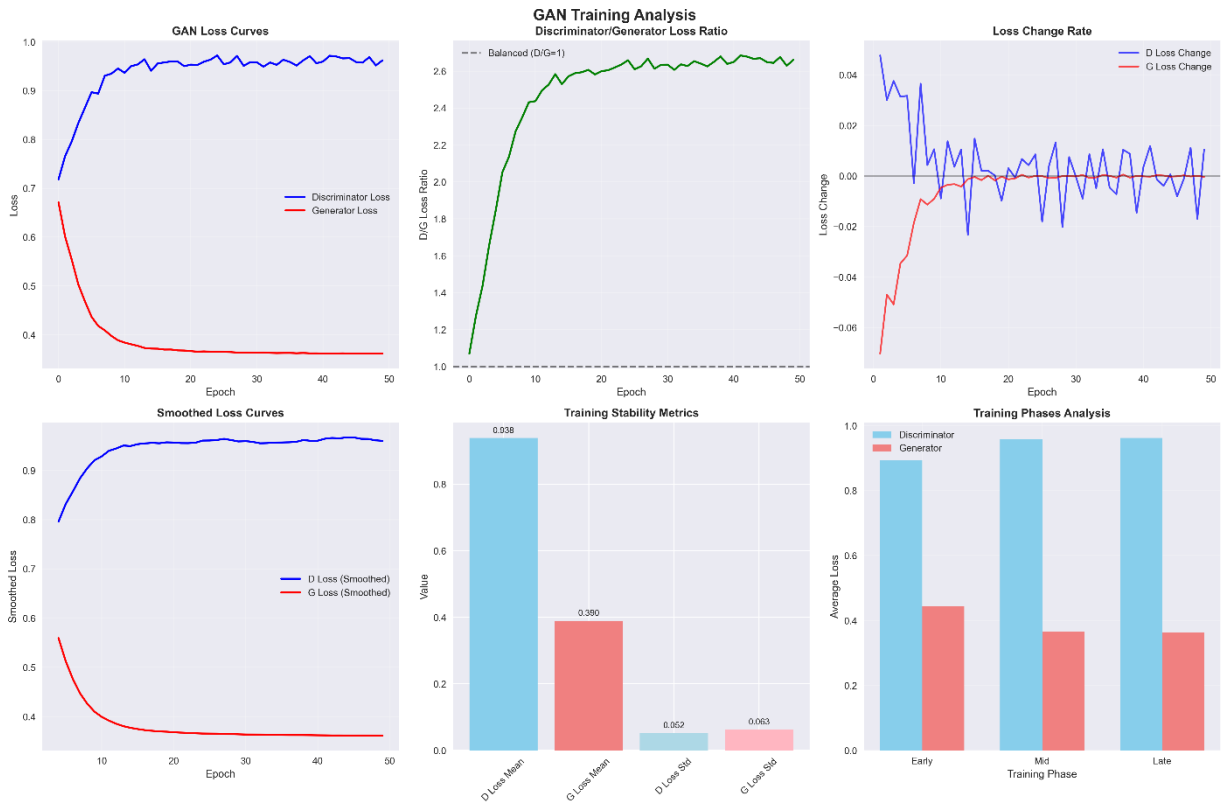


Figure 5.2 – GAN Training Analysis

As observed, the generator's loss gradually decreased, which indicated that the generator improved in producing synthetic samples. Meanwhile, the discriminator's loss increased and reached a steady value approximately close to a loss of 1.0. This suggests that the adversarial training has reached equilibrium, where neither network dominates, indicating that the synthetic data generation was successful. Following Figure 5.3 shows initial emotion distribution and amount of synthetic data generated to balance the dataset:

```
Emotion distribution comparison:
         Original  Balanced
emotion
happy         553       553
sad           338       553
hopeful       170       553
tense          13       553
excited         4       553

Balance improvement:
  Original balance ratio: 0.007
  Balanced balance ratio: 1.000
```

Figure 5.3 – Emotion Distribution Comparison: Original Vs. Balanced

5.2 Human Evaluation and Inter-rater Agreement

To ensure consistency on emotional perception among the participants, the survey followed the same 25 pop piano tracks taken from the EMOPIA dataset, which was analysed by the system as well.

Each of the 30 participants listened to all 25 tracks played by the author and categorised it into the six predefined emotional classes (*Happy*, *Excited*, *Tense*, *Fearful*, *Sad*, and *Hopeful*). Following table summarises the matrix of vote counts per emotion for each track, which will be used to calculate the inter-rater reliability.

Table 5.2 – Survey Results

Track	Happy	Excited	Tense	Hopeful	Fearful	Sad
Track-1-Q1__kJtgm1OUNA_0	7	6	4	10	3	0
Track-2-Q4_biROWEwkDQQ_1	8	0	0	7	0	15
Track-3-Q1_0vLPYiPN7qY_0	9	6	5	9	0	1
Track-4-Q4_FyK_c-TIcCA_1	12	3	4	7	1	3
Track-5-Q4_ADJ_PDauy-g_1	3	0	20	1	3	3
Track-6-Q2_jIKX3ShvLxo_2	3	8	13	4	1	1
Track-7-Q4_B3aRl8iTEKw_1	3	1	1	11	1	13
Track-8-Q1_6wFJhmhNeeg_0	15	5	0	8	0	2
Track-9-Q2_Dg935IbDggo_0	0	2	8	6	12	2
Track-10-Q2_9v2WSpn4FCw_4	2	14	8	2	3	1
Track-11-Q1__8v0MFBZoco_1	16	7	3	4	0	0
Track-12-Q4_7yW9c7t8Hq0_0	5	1	0	10	0	14
Track-13-Q2_08t73Qgjkt4_0	3	1	1	3	2	20
Track-14-Q3_veG92Oi-DlU_0	2	3	2	7	1	15
Track-15-Q1_5Ju9q1N2x0E_2	5	4	0	18	0	3
Track-16-Q3_vVTP0DOL_2Q_0	3	1	3	2	1	20
Track-17-Q3_wlOa1jkTn_U_2	0	2	5	5	7	11
Track-18-Q1__SJQaaRzD-A_1	13	5	0	10	0	2

Track-19-Q2_b8HVQtIoBYU_1	3	7	7	4	5	4
Track-20-Q4_9v2WSpn4FCw_9	3	1	2	8	4	12
Track-21-Q2_ANZf1QXsNrY_7	2	8	7	7	1	5
Track-22-Q1_2Z9SjI131jA_0	12	8	2	6	0	2
Track-23-Q3_7JIdJLkJ0S4_1	1	2	10	0	11	6
Track-24-Q3_Adil-zpJdEs_0	1	3	4	5	1	16
Track-25-Q3_2eIsQtm4YNs_1	3	6	1	8	1	11

5.2.1 Fleiss' Kappa Calculation

As defined in the Methodology chapter, the computation followed a five-step procedure as introduced by Fleiss (1971). This computation will produce:

- Proportion of raters per category Per item (P_{ij})
- Per-track agreement scores (P_i) (0 = Track-1)
- Mean observed agreement ($\bar{P}$): **0.295**
- Expected chance agreement ($\bar{P}_e$): **0.184**
- Overall Fleiss' Kappa (κ): **0.136**

As per the statistics, on average two participants had a 29.5% chance of agreeing on one emotional label on a given track whereas, 18.4% of that agreement could be explained by chance itself.

5.2.1.1 Per-track Agreement patterns

A range from 0.15 to 0.45 was observed from the per-track agreement values, showing that few pieces gained more consistent emotional responses that others.

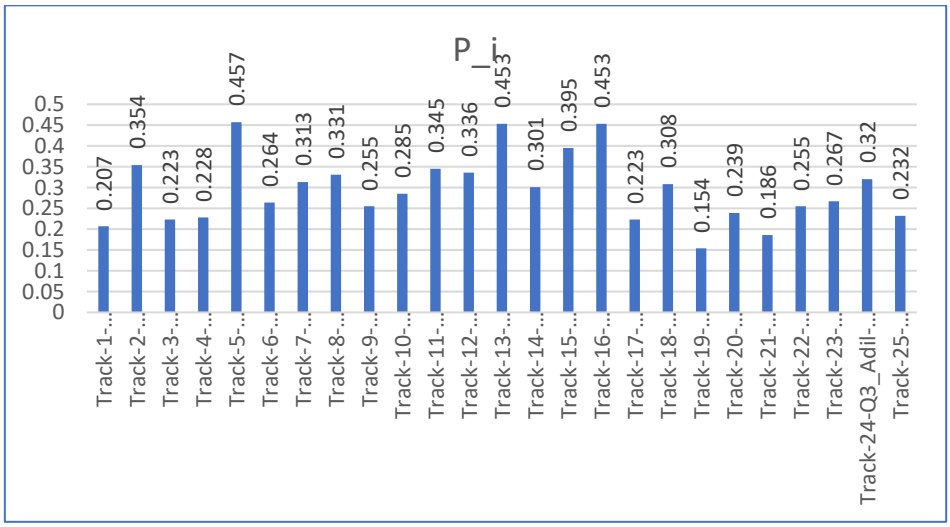


Figure 5.4 – Per track Agreement

As per the Figure 5.4, few tracks such as Track-5, Track-13 and Track-16 has reached high agreement levels (approximately P_i = 0.45), meanwhile, tracks such as Track-19 and Track-21 obtained lower agreement (approximately P_i = 0.15 – 0.18). This fluctuation shows that music emotion perception is subjective and different to each individual.

5.2.1.2 Distribution of Emotions

Following Figure illustrates the system output for Step 4: Category Proportions:

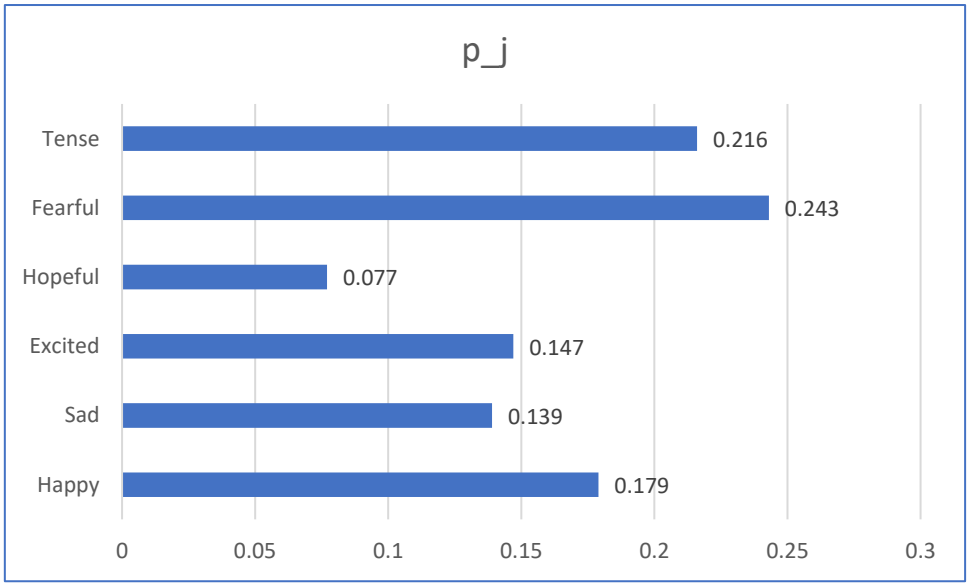


Figure 5.5 – Per category proportions

As observed, most participants frequently labelled *Sad* (24.3%) and *Hopeful* (21.6%) emotions. This is further followed by *Happy* (17.9%), *Excited* (13.9%) and finally *Fearful* (7.7%) being least commonly marked.

This uneven result suggests that most participants perceive EMOPIA's Pop Piano pieces as emotionally *Sad* and *Hopeful*, rather than high-arousal emotions such as fear or excitement.

5.2.1.3 Interpretation and Discussion

Following Figure shows the Fleiss' Kappa (κ) agreement output from the system:

```
STEP 5: Fleiss' Kappa (κ): 0.136
Interpretation: Slight agreement

Verification using statsmodels: κ = 0.136
```

Figure 5.6 – Fleiss Kappa agreement: System Output

The result κ = 0.136 shows that participants only slightly agreed more than what can be expected by random guessing. Thus, it can be concluded that emotional interpretation of these piano tracks was not uniform or consistent among the participants.

It is also evident that some tracks considered may have expressed more than one tone quality (as an example positive and melancholic)– leading some participants to perceive it as one emotion where others a different one (as an example *Sad* and *Hopeful*).

As per all results, research finding has two implications:

1. Reliability of Labels: Although the agreement is in slight, it shares a degree of understanding among the participants, meaning that human rating may still be valid despite the noisy ground-truth for evaluating the model.
2. Benchmark for System Evaluation: Despite the slight agreement, it still sets an upper limit to what can be potentially expected from the system. Therefore, mismatches between system outputs and survey outputs (later discussed in the chapter) can also be considered in the context of human variability than algorithmic error.

In conclusion, as per the Fleiss' Kappa analysis emotional perception of music is subjective yet patterned.

5.3 System Outputs and Emotion Classification Results

Table 5.2 presents the comparative analysis between human majority emotion (dominant emotion among the 30 participants in survey) and system's predicted emotion category for the same 25 tracks selected from EMOPIA dataset.

Table 5.2 – Comparison between Human Majority Emotion and System Prediction

Track	**Human Majority Emotion**	**System Prediction Category**
Track-1-Q1__kJtgm1OUNA_0	Hopeful	Hopeful
Track-2-Q4_biROWEwkDQQ_1	Sad	Happy
Track-3-Q1_0vLPYiPN7qY_0	Happy, Tense	Happy
Track-4-Q4_FyK_c-TIcCA_1	Happy	Happy
Track-5-Q4_ADJ_PDauy-g_1	Tense	Sad
Track-6-Q2_jIKX3ShvLxo_2	Tense	Sad
Track-7-Q4_B3aRl8iTEKw_1	Sad	Happy
Track-8-Q1_6wFJhmhNeeg_0	Happy	Tense
Track-9-Q2_Dg935IbDggo_0	Fearful	Hopeful

Track-10-Q2_9v2WSpn4FCw_4	Excited	Hopeful
Track-11-Q1__8v0MFBZoco_1	Happy	Happy
Track-12-Q4_7yW9c7t8Hq0_0	Sad	Happy
Track-13-Q2_08t73Qgjkt4_0	Sad	Sad
Track-14-Q3_veG92Oi-DlU_0	Sad	Sad
Track-15-Q1_5Ju9q1N2x0E_2	Hopeful	Happy
Track-16-Q3_vVTP0DOL_2Q_0	Sad	Sad
Track-17-Q3_wlOa1jkTn_U_2	Sad	Happy
Track-18-Q1__SJQaaRzD-A_1	Happy	Hopeful
Track-19-Q2_b8HVQtIoBYU_1	Excited, Tense	Hopeful
Track-20-Q4_9v2WSpn4FCw_9	Sad	Happy
Track-21-Q2_ANZf1QXsNrY_7	Excited	Happy
Track-22-Q1_2Z9SjI131jA_0	Happy	Happy
Track-23-Q3_7JIdJLkJ0S4_1	Fearful	Sad
Track-24-Q3_Adil-zpJdEs_0	Sad	Happy
Track-25-Q3_2eIsQtm4YNs_1	Sad	Sad

From the evaluated tracks, the system accurately matched 9 cases out of 25 (36% accuracy), reflecting a low accuracy level with regard to emotion perception. This discrepancy may be mainly due to two emotions out of the six emotions (*Fearful* and *Excited*) not being included in the overall classification, as mentioned previously (refer topic 5.1.1. Emotion Distribution).

The results show a clear bias towards *Happy* and *Sad* predication, which is in-line with EMOPIA's composition. Tracks such as Track-11-Q1__8v0MFBZoco_1, Track-4-Q4_FyK_c-TIcCA_1 and Track-14-Q3_veG92Oi-DlU_0 were accurately identified as *Happy* or *Sad* meanwhile Track-5-Q4_ADJ_PDauy-g_1 and Track-6-Q2_jIKX3ShvLxo_2 were misclassified as "Sad" despite participants perceiving high-arousal levels. These patterns suggest that high-valance, high-arousal tracks (*Happy*) and low-valance, low-arousal tracks (*Sad*) are identified by the system accurately.

However, the lack of research availability on emotion-based music feature thresholds still affects the overall accuracy of the system.

5.4 Recommendation System Evaluation

For each 25 evaluated query tracks, the system first displays the query emotion category (which is *Happy*, *Sad*, *Hopeful* or *Tense*), then returned the top-5 most similar EMOPIA soundtracks using cosine similarity.

Results per track can be observed below:

Track	Recommendations	System Prediction Category
Track-1-Q1__kJtgm1OUNA_0	Top 5 similar tracks: 1. Q3_FUAK5TBaNY8_2.mid (similarity: 0.891, emotion: hopeful) 2. Q3_wfXSdMsd4q8_1.mid (similarity: 0.859, emotion: hopeful) 3. Q3_wfXSdMsd4q8_4.mid (similarity: 0.833, emotion: hopeful) 4. Q4_ldCQ6N9G6Mk_0.mid (similarity: 0.801, emotion: hopeful) 5. Q3_FUAK5TBaNY8_3.mid (similarity: 0.796, emotion: hopeful)	Hopeful
Track-2-Q4_biROWEwkDQQ_1	Top 5 similar tracks: 1. Q4_HYVmgq5Y93g_0.mid (similarity: 0.860, emotion: happy) 2. Q4_V3Y9L4UOcpk_1.mid (similarity: 0.841, emotion: happy) 3. Q3_02t1bj7ZABY_2.mid (similarity: 0.835, emotion: happy) 4. Q3_Ie5koh4qvJc_14.mid (similarity: 0.812, emotion: happy) 5. Q3_Ie5koh4qvJc_15.mid (similarity: 0.810, emotion: happy)	Happy
Track-3-Q1_0vLPYiPN7qY_0	Top 5 similar tracks: 1. Q4_6kRPHamGDSo_2.mid (similarity: 0.884, emotion: happy) 2. Q1_KxlqB3j0zys_1.mid (similarity: 0.864, emotion: happy) 3. Q3_REq37pDAm3A_1.mid (similarity: 0.848, emotion: happy) 4. Q1_dfNdpy8TUzA_0.mid (similarity: 0.847, emotion: happy) 5. Q4_9eMxCs-LXCE_1.mid (similarity: 0.843, emotion: happy)	Happy
Track-4-Q4_FyK_c-TIcCA_1	Top 5 similar tracks: 1. Q4_UpHutdJvZMI_2.mid (similarity: 0.850, emotion: happy) 2. Q3_-vAz_HTFEXs_1.mid (similarity: 0.847, emotion: happy) 3. Q4_Ie5koh4qvJc_27.mid (similarity: 0.838, emotion: happy) 4. Q3_c6CwY8Gbw0c_4.mid (similarity: 0.838, emotion: happy) 5. Q3_PLfFWFZflQU_1.mid (similarity: 0.835, emotion: happy)	Happy
Track-5-Q4_ADJ_PDauy-g_1	Top 5 similar tracks: 1. Q4_DvIH5yUrboc_2.mid (similarity: 0.922, emotion: sad) 2. Q4_I2MrA-o5H8I_1.mid (similarity: 0.901, emotion: sad) 3. Q4_UaYYMRkHMYc_2.mid (similarity: 0.880, emotion: sad) 4. Q4_heBmBQDWj-M_1.mid (similarity: 0.872, emotion: sad) 5. Q4_I2MrA-o5H8I_2.mid (similarity: 0.868, emotion: sad)	Sad
Track-6-Q2_jIKX3ShvLxo_2	Top 5 similar tracks: 1. Q2_jIKX3ShvLxo_1.mid (similarity: 0.915, emotion: sad) 2. Q1_gSwv8hZGM-s_0.mid (similarity: 0.911, emotion: sad) 3. Q2_laInGhxT3PQ_1.mid (similarity: 0.898, emotion: sad) 4. Q1_qlbazHayULg_8.mid (similarity: 0.890, emotion: sad) 5. Q2_ItGNJM6skM4_1.mid (similarity: 0.866, emotion: sad)	Sad
Track-7-Q4_B3aRl8iTEKw_1	Top 5 similar tracks: 1. Q4_egYSmNuIFGk_0.mid (similarity: 0.893, emotion: happy) 2. Q4_pRuwHN-II44_0.mid (similarity: 0.880, emotion: happy) 3. Q4_Ie5koh4qvJc_32.mid (similarity: 0.865, emotion: happy) 4. Q3_fnqb8zqHqdY_0.mid (similarity: 0.863, emotion: happy) 5. Q4_FoTXpYZXxJs_0.mid (similarity: 0.858, emotion: happy)	Happy
Track-8-Q1_6wFJhmhNeeg_0	Top 5 similar tracks: 1. Q2_cm6E860vDjY_0.mid (similarity: 0.825, emotion: tense) 2. Q2_epX33OVpkmA_1.mid (similarity: 0.779, emotion: tense) 3. Q2_dtS02mrDMsM_2.mid (similarity: 0.719, emotion: tense) 4. Q2_UE0y8MHqT-g_2.mid (similarity: 0.689, emotion: tense) 5. Q2_JMt4m608s3k_2.mid (similarity: 0.534, emotion: tense)	Tense
Track-9-Q2_Dg935IbDggo_0	Top 5 similar tracks: 1. Q1_jziI_cuN_H8_0.mid (similarity: 0.753, emotion: hopeful) 2. Q1_UOSlDydo94E_0.mid (similarity: 0.724, emotion: hopeful) 3. Q3_G4rL_OtfFAU_0.mid (similarity: 0.724, emotion: hopeful) 4. Q2_ANZf1QXsNrY_6.mid (similarity: 0.700, emotion: hopeful) 5. Q3_TYkTOwBfFB8_0.mid (similarity: 0.679, emotion: hopeful)	Hopeful
Track-10-Q2_9v2WSpn4FCw_4	Top 5 similar tracks: 1. Q2_9v2WSpn4FCw_6.mid (similarity: 0.960, emotion: hopeful) 2. Q2_0v2N1ROvEI0_0.mid (similarity: 0.946, emotion: hopeful) 3. Q2_qlbazHayULg_0.mid (similarity: 0.920, emotion: hopeful) 4. Q3_uj3Gif77SYM_8.mid (similarity: 0.870, emotion: hopeful) 5. Q2_k-FNDbK6Qhg_0.mid (similarity: 0.843, emotion: hopeful)	Hopeful

Track-11-Q1__8v0MFBZoco_1	Top 5 similar tracks: 1. Q2_Q5b5unyP8BM_0.mid (similarity: 0.906, emotion: happy) 2. Q2_7n32BtYb4d4_2.mid (similarity: 0.904, emotion: happy) 3. Q2_7n32BtYb4d4_0.mid (similarity: 0.896, emotion: happy) 4. Q1__8v0MFBZoco_0.mid (similarity: 0.896, emotion: happy) 5. Q1_e838BE_MH6s_4.mid (similarity: 0.894, emotion: happy)	Happy
Track-12-Q4_7yW9c7t8Hq0_0	Top 5 similar tracks: 1. Q4_vBHoSz8cJIE_4.mid (similarity: 0.928, emotion: happy) 2. Q4_Tc684nD8CEA_1.mid (similarity: 0.927, emotion: happy) 3. Q3_MbvPxCUMSek_2.mid (similarity: 0.917, emotion: happy) 4. Q3_QJlnTN7HRwE_1.mid (similarity: 0.904, emotion: happy) 5. Q4_NP0lwB-_-og_0.mid (similarity: 0.903, emotion: happy)	Happy
Track-13-Q2_08t73Qgjkt4_0	Top 5 similar tracks: 1. Q4_1vjy9oMFa8c_0.mid (similarity: 0.870, emotion: sad) 2. Q3_2v6gJi03LlA_0.mid (similarity: 0.850, emotion: sad) 3. Q2_1kny88W533Q_0.mid (similarity: 0.849, emotion: sad) 4. Q3_E5qEloUO3SM_1.mid (similarity: 0.842, emotion: sad) 5. Q3_PXOWy7NiZhk_0.mid (similarity: 0.834, emotion: sad)	Sad
Track-14-Q3_veG92Oi-DlU_0	Top 5 similar tracks: 1. Q3_g-yM0Lsp4lc_0.mid (similarity: 0.965, emotion: sad) 2. Q4_mGYKWCVnMwQ_0.mid (similarity: 0.940, emotion: sad) 3. Q4_xgtwQGeB6_0_0.mid (similarity: 0.923, emotion: sad) 4. Q3_TonQX8XbvX8_2.mid (similarity: 0.892, emotion: sad) 5. Q4_mGYKWCVnMwQ_1.mid (similarity: 0.889, emotion: sad)	Sad
Track-15-Q1_5Ju9q1N2x0E_2	Top 5 similar tracks: 1. Q1_Ie5koh4qvJc_6.mid (similarity: 0.852, emotion: happy) 2. Q4_ldCQ6N9G6Mk_5.mid (similarity: 0.812, emotion: happy) 3. Q1_yFw_kO7DF-Y_2.mid (similarity: 0.768, emotion: happy) 4. Q3_bbU31JLtlug_1.mid (similarity: 0.765, emotion: happy) 5. Q2_CxFXrYgdSSI_2.mid (similarity: 0.758, emotion: happy)	Happy
Track-16-Q3_vVTP0DOL_2Q_0	Top 5 similar tracks: 1. Q3_vVTP0DOL_2Q_3.mid (similarity: 0.985, emotion: sad) 2. Q3_vVTP0DOL_2Q_1.mid (similarity: 0.969, emotion: sad) 3. Q3_wfXSdMsd4q8_0.mid (similarity: 0.951, emotion: sad) 4. Q3_K84TcgjCRt4_0.mid (similarity: 0.920, emotion: sad) 5. Q3_gvWDOIiocuE_2.mid (similarity: 0.899, emotion: sad)	Sad
Track-17-Q3_wlOa1jkTn_U_2	Top 5 similar tracks: 1. Q2_S1T3UF1vhSk_1.mid (similarity: 0.906, emotion: happy) 2. Q1_60LLKmpgzRM_0.mid (similarity: 0.894, emotion: happy) 3. Q3_ilCcZpD39HU_2.mid (similarity: 0.894, emotion: happy) 4. Q4_NP0lwB-_-og_0.mid (similarity: 0.887, emotion: happy) 5. Q4_d_49EtXDMFE_2.mid (similarity: 0.878, emotion: happy)	Happy
Track-18-Q1__SJQaaRzD-A_1	Top 5 similar tracks: 1. Q1_pNwQ9Tu_bCs_2.mid (similarity: 0.886, emotion: hopeful) 2. Q1_pNwQ9Tu_bCs_0.mid (similarity: 0.859, emotion: hopeful) 3. Q1_zmZHNy9T8Pg_2.mid (similarity: 0.847, emotion: hopeful) 4. Q2_XC_SiJszQx0_1.mid (similarity: 0.840, emotion: hopeful) 5. Q1_8rupdevqfuI_0.mid (similarity: 0.832, emotion: hopeful)	Hopeful
Track-19-Q2_b8HVQtIoBYU_1	Top 5 similar tracks: 1. Q2_P-We4q3goKU_5.mid (similarity: 0.785, emotion: hopeful) 2. Q2_x5ibvz38jOs_1.mid (similarity: 0.782, emotion: hopeful) 3. Q2_cQp1BYDGcRo_0.mid (similarity: 0.674, emotion: hopeful) 4. Q1_dfNdpy8TUzA_1.mid (similarity: 0.649, emotion: hopeful) 5. Q1_8rupdevqfuI_0.mid (similarity: 0.629, emotion: hopeful)	Hopeful
Track-20-Q4_9v2WSpn4FCw_9	Top 5 similar tracks: 1. Q1_T92R7xjce34_1.mid (similarity: 0.924, emotion: happy) 2. Q3_ltxNPJda7zE_2.mid (similarity: 0.846, emotion: happy) 3. Q1_9v2WSpn4FCw_5.mid (similarity: 0.840, emotion: happy) 4. Q1_9v2WSpn4FCw_0.mid (similarity: 0.789, emotion: happy) 5. Q2_epX33OVpkmA_2.mid (similarity: 0.739, emotion: happy)	Happy

Track-21-Q2_ANZf1QXsNrY_7	Top 5 similar tracks: 1. Q1_NGE9ynTJABg_0.mid (similarity: 0.852, emotion: happy) 2. Q2_cm6E860vDjY_1.mid (similarity: 0.844, emotion: happy) 3. Q1_7yW9c7t8Hq0_1.mid (similarity: 0.834, emotion: happy) 4. Q1_jziI_cuN_H8_2.mid (similarity: 0.831, emotion: happy) 5. Q2_qlbazHayULg_6.mid (similarity: 0.813, emotion: happy)	Happy
Track-22-Q1_2Z9SjI131jA_0	Top 5 similar tracks: 1. Q4_gxsdWKc1QaM_0.mid (similarity: 0.880, emotion: happy) 2. Q4_iHPKusssXzk_0.mid (similarity: 0.870, emotion: happy) 3. Q1_2Z9SjI131jA_12.mid (similarity: 0.868, emotion: happy) 4. Q2_5CEAeMiXKaA_0.mid (similarity: 0.865, emotion: happy) 5. Q4_k6aLLVX1D7Y_1.mid (similarity: 0.860, emotion: happy)	Happy
Track-23-Q3_7JIdJLkJ0S4_1	Top 5 similar tracks: 1. Q1_im4Qxn3GQvo_0.mid (similarity: 0.858, emotion: sad) 2. Q2_Ie5koh4qvJc_10.mid (similarity: 0.842, emotion: sad) 3. Q3_wlOa1jkTn_U_0.mid (similarity: 0.809, emotion: sad) 4. Q1_GbUV3TXUzeQ_2.mid (similarity: 0.801, emotion: sad) 5. Q1_1Qc15G0ZHIg_1.mid (similarity: 0.797, emotion: sad)	Sad
Track-24-Q3_Adil-zpJdEs_0	Top 5 similar tracks: 1. Q3__vZOEQCYSaY_2.mid (similarity: 0.904, emotion: happy) 2. Q3_K84TcgjCRt4_2.mid (similarity: 0.892, emotion: happy) 3. Q4_HYVmgq5Y93g_0.mid (similarity: 0.890, emotion: happy) 4. Q3_2v6gJi03LlA_1.mid (similarity: 0.874, emotion: happy) 5. Q3_TYkTOwBfFB8_1.mid (similarity: 0.869, emotion: happy)	Happy
Track-25-Q3_2eIsQtm4YNs_1	Top 5 similar tracks: 1. Q4_bfopzItCYrE_2.mid (similarity: 0.858, emotion: sad) 2. Q1_K6OFDxBU370_0.mid (similarity: 0.825, emotion: sad) 3. Q4_fOYX0uH8mSQ_0.mid (similarity: 0.821, emotion: sad) 4. Q4_a5QCcwEjxAk_0.mid (similarity: 0.820, emotion: sad) 5. Q4_JxSU49jFKwM_1.mid (similarity: 0.803, emotion: sad)	Sad

Across all queries, it can be overserved that top-5 emotion consistencies = 100% per track (i.e., 5/5 correct for every query file). This suggested that feature representations used for similarity are in-line with emotion categories produced by the *EmotionClassifier.*

Most emotions display tight neighbourhoods in terms of similarity (such as *Happy*/*Sad*/*Hopeful* with a range of 0.6 – 0.8) while, *Tense* shows a wider dispersion (as an example, Track-8-Q1_6wFJhmhNeeg_0) with a lower boundary close to 0.53. This suggests that *Tense* category may not share consistent musical patterns as other categories.

Further, it can be observed that few recommendations include different segments of the same track. This is expected and it suggests that similarity recommendation given are accurate.

5.5 Summary

Through this chapter, the overall performance of the GAN-based MER and recommendation system was evaluated in two main areas, namely, human evaluation and agreement, system-generated results. Fleiss Kappa of 0.136 was obtained by having 30 participants rating 25 EMOPIA tracks in the human evaluation, showing that there is minimal agreement among the participants.

The system was compared with the majority emotions identified by participants and had achieved an overall accuracy of 36% (9 out of 25 matches), matching the emotions contained in the dataset best *Happy*, *Sad*, *Hopeful* and *Tense*. The prediction of *Excited* and *Fearful* was not made since EMOPIA dataset did not contain MIDI files that agree with the feature thresholds defined on these emotions.

There was more consistency in the recommendation aspect as it obtained 100% on Top-5 accuracy: all of the retrieved tracks were similar in terms of emotion class of the input track, and similarity scores were predominantly above 0.8. These findings indicate that balancing by using GAN can enhance emotional clustering and quality of recommendation. Nonetheless, the subjective human variability and the scarcity of emotional variety within the data are still issues that would be improved in the future.

6 Discussion and Conclusion

This chapter critically discusses the outcomes of the developed system, understanding the results produced in the previous chapter while reflecting on the limitations and possible future suggestions of this research. The discussion explores on how the extracted music features relate to emotion perception, variability in human response, contribution of GAN towards solving data imbalance, and other gaps that remains unaddressed.

This chapter concludes with potential recommendations for future research and improvements to increase the robustness and emotional sensitivity of similar systems.

6.1 Discussion and Key Findings

6.1.1 Emotional feature thresholds and lack of research correlation

One of the major challenges encountered during this research was the lack of well-defined feature thresholds linking MIDI feature or acoustic features to human emotions. Features such as *pitch_min, pitch_max, velocity_min, velocity_max, tempo_mean, tempo_std, chord_changes, harmony_complexity, rhythm_mean, rhythm_std, syncopation, note_overlap_ratio, total_notes, tempo_variation, dynamic_range,* and *dynamic_contrast* that were extracted through the system could not be utilized due to lack of existing literature providing quantitative values on how these features associate with particular emotions.

Although previous work has established general relationships such as higher pitch and tempo is associated with happiness or excitement and lower pitch along with slower tempo align with sadness or calmness, specific numeric thresholds are inconsistent. Due to this gap, defining precise threshold boundaries for each emotional category was difficult. The *EmotionClassifier* was thus forced to assign emotional categories to continuous increments and not discrete boundaries causing some overlaps between categories of *Happy*, *Hopeful* and *Sad*.

6.1.2 Variability in human emotional perception

The survey proved that the perception of the emotion of music is subjective in nature. On the same 25 tracks, there was only slight agreement (Fleiss $\kappa = 0.136$) which showed that there was a high level of variability among perceived emotion. This result is consistent with previous research by Eerola & Vuoskoski (2013) who demonstrated that personality, mood, and cultural background have a significant influence on emotional judgments of the same musical piece. As

an example, a major-key composition, moderate tempo, can give one listener hopefulness, and to another, happiness or calmness. This diversity in human perception makes it understandable that even highly trained models would only be able to reach moderate degrees of compatibility with human ratings. It also emphasises the need to include listener-specific adaptation or personalised emotion models in the future recommendation systems.

6.1.3 Effectiveness of GAN-based data augmentation

As per the research, the usage of GAN to solve data imbalance among emotion categories had been successful. The GAN was able to balance the EMOPIA dataset in four emotional groups including Happy, Sad, Hopeful and Tense by synthesising samples using data that maintained key musical feature distributions. This created balanced dataset resulted in better clustering of emotion and a steady Top-5 recommendation accuracy of 100%. This result supports the results of previous experiments, including Bao et al., (2021) and Dong et al., (2017), who demonstrated that GAN-based augmentation improves the performance in feature recognition in the situation when labelled data is scarce.

6.1.4 Comparison between EMOPIA classification and System classification

One weakness of this study is that there was no direct comparative analysis between the original emotion classification of the EMOPIA dataset and the scores that were achieved by the developed system. EMOPIA uses valence-arousal dimensions to make quadrant-based labels, but this study is a re-mapped emotional category into six discrete categories. It would have been more effective to draw a direct comparison as to how well the proposed system is able to enhance or break with the original labelling framework of EMOPIA. Future studies must hence consider performance based on standard measures of accuracy, precision, and F1-score relative to the performance on EMOPIA at its baseline thus quantifying the improvement that GAN augmentation and feature-based classification make.

6.1.5 Verification of emotion-based recommendations produced by the system

Despite this recommendation system having a 100% emotion similarity, it was evaluated solely through algorithms and not by humans. To have a real validation, the participants would have to listen to the suggested tracks and state if the same emotions were perceived as the input. Emotion-based recommendation systems require human-centred evaluation as suggested by Y.-H. Yang & Chen (2012) since the similarity based on emotion cannot be explained entirely through the application of numerical values. Hence, the system proves to be technically accurate

in clustering, but its emotional relevance has not been proved to be accurate. Future studies need to utilise the listener studies or pairwise emotion-similarity ratings to ensure that system-suggested tracks trigger a similar emotional reaction.

6.2 Conclusion

This study aimed to investigate the possibilities of GANs to improve MER and emotion-based recommendation systems with the help of the EMOPIA Pop Piano dataset. The overall objective was to investigate the possibility of GAN data enhancement to enhance the accuracy and reliability of emotion recognition and recommendation, to fill the gap between computational emotion recognition and human perception of emotions.

This research started by conducting a detailed review of the current MER systems, their main methodologies, features and shortcomings. Earlier publications were able to make major advancements in audio feature extraction and emotion mapping, but issues such as data imbalance, emotion subjectivity, and lack of standardised thresholds used in the classification of emotions were still present. This informed the research direction and rationalised the use of GAN-based data augmentation as a mechanism to sample out emotion in datasets. The initial objective revealed that current MER resources, despite the effectiveness in the detection of basic emotion conditions, failed to generalise between genres and emotional variation, which is the primary weakness of the given project.

The second goal aimed at reviewing previous studies in which GANs have been employed in data augmentation in other similar fields like speech emotion recognition and EEG-based emotion recognition. These experiments were able to verify that GANs are capable of producing realistic and high-quality synthetic data, supporting the hypothesis that GANs can enhance the performance of music emotion classification. Based on this, a simple GAN architecture was used to synthesise underrepresented emotions to the EMOPIA dataset. This procedure was effective to balance the dataset, which enabled more consistent training of the classifiers and less emotional bias, which supports Hypothesis H1 which stated that GAN-based augmentation improves classification performance in comparison to training on the original EMOPIA dataset only.

The emotion classification model, which was constructed on the basis of MIDI-based features, utilised research-driven thresholds based on pitch, velocity and tempo to classify the emotion into six categories: *happy*, *sad*, *excited*, *hopeful*, *tense* and *fearful*. Nevertheless, since the EMOPIA dataset is composed in such a way, it was possible to consider only four emotions

(*happy, sad, hopeful*, and *tense*), which was strongly represented, and *excited* and *fearful* appeared to reach feature thresholds infrequently. This was a partial rejection of Hypothesis H2 as though it was true that MIDI feature of emotion classification was interactively interpretable and effective, the lack of corresponding audio-based features limited comparative analysis. This hypothesis can be fully tested by having a hybrid feature fusion in future work.

The third hypothesis (H3) was that musical features correlate directly with the human emotion perception. This was partly verified by the 30 participants survey, which showed that human beings have a moderate level of agreement with the system on what should be classified in a specific category based on the tracks played. Nevertheless, a Kappa analysis conducted by Fleiss showed that the agreement between participants was rather weak, which was a consequence of the subjectivity of the process of emotional interpretation in music and not the disadvantage of the system itself.

The last hypothesis (H4) tested whether emotion-based recommendations were consistent. The recommendation module relied on cosine similarity to identify tracks that had similar feature structures to produce 100% intra-emotion consistency, that is, all five tracks recommended with each query had the same predicted emotion. Although, this establishes robust consistency at the system level, the similarity of emotions perceived should be determined by human beings. This is still to be done suggesting future experiments should incorporate listener feedback.

In conclusion, the use of GANs in the given research was effective to enhance the size and quality of emotional music datasets by adding balance and classifier confidence. The system achieved its main goals and confirmed the application of symbolic features in the perception of emotions. But it also noted continuing difficulties – especially that the thresholds of emotions are not standardised, and that emotional perception is necessarily subjective. The results help further the development of emotion-sensitive recommendation systems and provide a basis upon which emotion-oriented work in music technology can continue to be done in the future with the consideration of human evaluation and hybrid data representation.

References

Bao, G., Yan, B., Tong, L., Shu, J., Wang, L., Yang, K., & Zeng, Y. (2021). Data Augmentation for EEG-Based Emotion Recognition Using Generative Adversarial Networks. *Frontiers in Computational Neuroscience*, *15*. https://doi.org/10.3389/fncom.2021.723843

Bertin-Mahieux, T., Ellis, D., Whitman, B., & Lamere, P. (2011). The Million Song Dataset. *Proceedings of the 12th International Conference on Music Information Retrieval (ISMIR 2011)*, 591–596.

Chang, X., Zhang, X., Zhang, H., & Ran, Y. (2024). *Music Emotion Prediction Using Recurrent Neural Networks* (No. arXiv:2405.06747). arXiv. https://doi.org/10.48550/arXiv.2405.06747

Cheng, D. S., Joachims, T., & Turnbull, D. (2020). Exploring Acoustic Similarity for Novel Music Recommendation. In *Proceedings of the First MiniCon Conference*. Ismir 2020 Virtual Conference. https://program.ismir2020.net/poster_4-15.html

Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, *21*(1), 6. https://doi.org/10.1186/s12864-019-6413-7

Choi, K., Fazekas, G., Sandler, M., & Cho, K. (2016). *Convolutional Recurrent Neural Networks for Music Classification* (No. arXiv:1609.04243). arXiv. https://doi.org/10.48550/arXiv.1609.04243

christianversloot. (n.d.). *Machine-learning-articles/implementing-relu-sigmoid-and-tanh-in-keras.md at main · christianversloot/machine-learning-articles*. GitHub. Retrieved 6 October 2025, from https://github.com/christianversloot/machine-learning-articles/blob/main/implementing-relu-sigmoid-and-tanh-in-keras.md

Dixon, S., Gómez, E., & Volk, A. (2018). *Editorial: Introducing the Transactions of the International Society for Music Information Retrieval | Transactions of the International Society for Music Information Retrieval*. https://doi.org/10.5334/tismir.22

Dong, H.-W., Hsiao, W.-Y., Yang, L.-C., & Yang, Y.-H. (2017). *MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment* (No. arXiv:1709.06298). arXiv. https://doi.org/10.48550/arXiv.1709.06298

Downie, J. S. (2003). Music information retrieval. *Annual Review of Information Science and Technology*, *37*(1), 295–340. https://doi.org/10.1002/aris.1440370108

Eck, D., & Schmidhuber, J. (n.d.). *A First Look at Music Composition using LSTM Recurrent Neural Networks*.

Editorial: Introducing the Transactions of the International Society for Music Information Retrieval | Transactions of the International Society for Music Information Retrieval. (n.d.). Retrieved 2 October 2025, from https://transactions.ismir.net/articles/10.5334/tismir.22

Eerola, T., & Anderson, C. (2025). *A Meta-Analysis of Music Emotion Recognition Studies*. OSF. https://doi.org/10.31234/osf.io/5azc9_v1

Eerola, T., & Vuoskoski, J. K. (2013). A Review of Music and Emotion Studies: Approaches, Emotion Models, and Stimuli. *Music Perception*, *30*(3), 307–340. https://doi.org/10.1525/mp.2012.30.3.307

Fernández-Sotos, A., Fernández-Caballero, A., & Latorre, J. M. (2016). Influence of Tempo and Rhythmic Unit in Musical Emotion Regulation. *Frontiers in Computational Neuroscience*, *10*. https://doi.org/10.3389/fncom.2016.00080

Fleiss, J. L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, *76*(5), 378–382. https://doi.org/10.1037/h0031619

Ghias, A., Logan, J., Chamberlin, D., & Smith, B. C. (1995). Query by humming: Musical information retrieval in an audio database. *Proceedings of the Third ACM International Conference on Multimedia - MULTIMEDIA '95*, 231–236. https://doi.org/10.1145/217279.215273

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). *Generative Adversarial Networks* (No. arXiv:1406.2661). arXiv. https://doi.org/10.48550/arXiv.1406.2661

Hevner, K. (1936). Experimental Studies of the Elements of Expression in Music. *The American Journal of Psychology*, *48*(2), 246–268. https://doi.org/10.2307/1415746

Hung, H.-T., Ching, J., Doh, S., Kim, N., Nam, J., & Yang, Y.-H. (2021). *EMOPIA: A Multi-Modal Pop Piano DatasetFor Emotion Recognition and Emotion-based Music Generation* (No. arXiv:2108.01374). arXiv. https://doi.org/10.48550/arXiv.2108.01374

Juslin, P. N., & Laukka, P. (2004). Expression, Perception, and Induction of Musical Emotions: A Review and a Questionnaire Study of Everyday Listening. *Journal of New Music Research*, *33*(3), 217–238. https://doi.org/10.1080/0929821042000317813

Juslin, P. N., & Västfjäll, D. (2008). Emotional responses to music: Theneed to consider underlying mechanisms. *Behavioral and Brain Sciences*,*31*(5), 559–575. https://doi.org/10.1017/S0140525X08005293

Juslin, P., & Sloboda, J. (2013). Music and Emotion. In *The Psychology of Music* (pp. 583–645). https://doi.org/10.1016/B978-0-12-381460-9.00015-8

Kang, J., & Herremans, D. (2025). *Are We There Yet? A Brief Survey of Music Emotion Prediction Datasets, Models and Outstanding Challenges* (No. arXiv:2406.08809). arXiv. https://doi.org/10.48550/arXiv.2406.08809

Krumhansl, C. L. (1997). An exploratory study of musical emotions and psychophysiology. *Canadian Journal of Experimental Psychology / Revue Canadienne de Psychologie Expérimentale*, *51*(4), 336–353. https://doi.org/10.1037/1196-1961.51.4.336

Landis, J. R., & Koch, G. G. (1977). The Measurement of ObserverAgreement for Categorical Data. *Biometrics*, *33*(1), 159. https://doi.org/10.2307/2529310

Lartillot, O., & Toiviainen, P. (n.d.). *MIR IN MATLAB (II): A TOOLBOX FOR MUSICAL FEATURE EXTRACTION FROM AUDIO*.

Latif, S., Rana, R., Khalifa, S., Jurdak, R., & Epps, J. (2020). *Direct Modelling of Speech Emotion from Raw Speech* (No. arXiv:1904.03833). arXiv. https://doi.org/10.48550/arXiv.1904.03833

Li, Y., Li, X., Lou, Z., & Chen, C. (2022). Long Short-Term Memory-Based Music Analysis System for Music Therapy. *Frontiers in Psychology*, *13*, 928048. https://doi.org/10.3389/fpsyg.2022.928048

Lin, Y., Chen, X., & Yang, D. (2013). *Exploration of Music Emotion Recognition Based on MIDI*. International Society for Music Information Retrieval Conference. https://www.semanticscholar.org/paper/Exploration-of-Music-Emotion-Recognition-Based-on-Lin-Chen/96adf37288a58596c9964742f33cb6e44b8a7654

Neves, P. L. T., Fornari, J., & Florindo, J. B. (2022). *GENERATING MUSIC WITH SENTIMENT USING TRANSFORMER-GANS*.

P.E., N.-K., Onu, F., A.U., D., Igwe, J., & N.H., O. (2023). Detailed Study of the Object-Oriented Programming (OOP) Features in Python. *British Journal of Computer, Networking and Information Technology*, *6*, 83–93. https://doi.org/10.52589/BJCNIT-FACSOJAO

Poli, G. D., Mion, L., & Rapanà, E. (2007). *Exploring similarities of affective and sensorial expressive intentions in music performance*. https://api.semanticscholar.org/CorpusID:148740135

Priya, L. K., Nanduri, A. S. R., Mrudula, D., Golla, H., & Pradeep, R. (2025). Review on Emotion-Centric Music Recommender Systems. In S. M. Thampi, V. Chaudhary, A.-S. K. Pathan, K. Ching Li, & D. Krishnaswamy (Eds), *Fifth International Conference on*

Computing and Network Communications (pp. 117–128). Springer Nature. https://doi.org/10.1007/978-981-97-4711-5_9

Qiao, Y., Mu, J., Xie, J., Hu, B., & Liu, G. (2024). Music emotion recognition based on temporal convolutional attention network using EEG. *Frontiers in Human Neuroscience*, *18*, 1324897. https://doi.org/10.3389/fnhum.2024.1324897

Raffel, C., McFee, B., Humphrey, E. J., Salamon, J., Nieto, O., Liang, D., & Ellis, D. (2014). *MIR_EVAL: A Transparent Implementation of Common MIR Metrics*. International Society for Music Information Retrieval Conference. https://www.semanticscholar.org/paper/MIR_EVAL:-A-Transparent-Implementation-of-Common-Raffel-McFee/6d37fbd2fccaef3ecdf75d34a7aee18ab9519a6f

Russell, J. (1980). A Circumplex Model of Affect. *Journal of Personality and Social Psychology*, *39*, 1161–1178. https://doi.org/10.1037/h0077714

Schedl, M., Zamani, H., Chen, C.-W., Deldjoo, Y., & Elahi, M. (2018). Current Challenges and Visions in Music Recommender Systems Research. *International Journal of Multimedia Information Retrieval*, *7*(2), 95–116. https://doi.org/10.1007/s13735-018-0154-2

Siles, I., Segura-Castillo, A., Sancho, M., & Solís-Quesada, R. (2019). Genres as Social Affect: Cultivating Moods and Emotions through Playlists on Spotify. *Social Media + Society*. https://doi.org/10.1177/2056305119847514

Soleymani, M., Aljanaki, A., & Yang, Y.-H. (n.d.). *DEAM: MediaEval Database for Emotional Analysis in Music*.

Thayer, R. E. (1989). *The Biopsychology of Mood and Arousal*. Oxford University Press USA.

Tran, H., Le, T., Do, A., Vu, T., Bogaerts, S., & Howard, B. (2023). Emotion-Aware Music Recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence*, *37*(13), 16087–16095. https://doi.org/10.1609/aaai.v37i13.26911

Wang, A. L.-C. (n.d.). *An Industrial-Strength Audio Search Algorithm*.

Yang, L.-C., Chou, S.-Y., & Yang, Y.-H. (n.d.). *MIDINET: A CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORK FOR SYMBOLIC-DOMAIN MUSIC GENERATION*.

Yang, Y.-H., & Chen, H. H. (2012). Machine Recognition of Music Emotion: A Review. *ACM Trans. Intell. Syst. Technol.*, *3*(3), 40:1-40:30. https://doi.org/10.1145/2168752.2168754

Zentner, M., Grandjean, D., & Scherer, K. R. (2008). Emotions evoked by the sound of music: Characterization, classification, and measurement. *Emotion*, *8*(4), 494–521. https://doi.org/10.1037/1528-3542.8.4.494

Appendices

Appendix A – Code

Main.py

```
"""
Full code found on GitHub: https://github.com/katp1998/mr-system
"""

import os
import pandas as pd
import numpy as np
from feature_extractor import MIDIFeatureExtractor
from emotion_classifier import EmotionClassifier
from simple_gan import SimpleGAN
from recommendation_system import RecommendationSystem
from eda_analysis import EDAAnalyzer

def main():
    print("Music Emotion Analysis and Recommendation System")
    print("=" * 50)

    # Setup paths
    midi_dir = "../EMOPIA_1.0/midis"
    output_dir = "results"
    os.makedirs(output_dir, exist_ok=True)

    # 1: Extract features from MIDI files
    print("\nExtracting features from MIDI files...")
    extractor = MIDIFeatureExtractor()

    features_list = []
    midi_files = [f for f in os.listdir(midi_dir) if f.endswith('.mid')]

    print(f"Found {len(midi_files)} MIDI files")

    for i, midi_file in enumerate(midi_files):  # Iterate over all files
        if i % 100 == 0:
            print(f"Processing file {i+1}/{len(midi_files)}")

        midi_path = os.path.join(midi_dir, midi_file)
        features = extractor.extract_features(midi_path)

        if features:
            features_list.append(features)

    # Convert to DataFrame
```

```
    df = pd.DataFrame(features_list)
    print(f"Extracted features from {len(df)} files")

    # Classify emotions
    print("\n2. Classifying emotions...")
    classifier = EmotionClassifier()
    df = classifier.classify_dataset(df)

    eda = EDAAnalyzer(output_dir)

    # Show emotion distribution
    emotion_dist = classifier.get_emotion_distribution(df)
    print("Emotion distribution:")
    print(emotion_dist)

    # Save initial results
    df.to_csv(f"{output_dir}/initial_features.csv", index=False)

    # 3: Balance dataset with GAN
    print("\nBalancing dataset with GAN...")

    # Prepare features for GAN
    numeric_features = df.select_dtypes(include=[np.number]).drop(['emotion'],
axis=1, errors='ignore')
    feature_dim = len(numeric_features.columns)

    # Initialize and train GAN
    gan = SimpleGAN(feature_dim=feature_dim)
    gan.train(df, epochs=50, eda_analyzer=eda)  # Pass EDA analyzer for loss
tracking

    # Balance dataset to match the largest category (happy = 553)
    balanced_df = gan.balance_dataset(df, target_samples_per_emotion=553)

    # Save balanced dataset
    # balanced_df.to_csv(f"{output_dir}/balanced_dataset.csv", index=False)

    # EDA on balanced dataset and comparison
    print("\nExploratory Data Analysis (Balanced Dataset)...")
    eda.analyze_dataset(balanced_df, "Balanced Dataset (After GAN)")
    eda.compare_datasets(df, balanced_df)

    # Model training visualizations
    print("\nModel Training Visualizations...")
    eda.visualize_training()

    # Train recommendation system
    print("\nraining recommendation system...")
    rec_system = RecommendationSystem()
```

```
    rec_system.train(balanced_df)

    # Show system stats
    stats = rec_system.get_emotion_stats()
    print("Recommendation system stats:")
    print(stats)

    # 5: Test recommendation system
    print("\nTesting recommendation system...")

    # Find a query file
    if len(df) > 0:

        target_file = "Q1_0vLPYiPN7qY_0.mid" # CHANGABLE VARIABLE!!
        query_row = df[df['file_path'].str.contains(target_file)]

        if len(query_row) > 0:
            query_file = query_row.iloc[0]['file_path']
            query_emotion = query_row.iloc[0]['emotion']
        else:
            # Fallback to first file if target not found
            query_file = df.iloc[0]['file_path']
            query_emotion = df.iloc[0]['emotion']
        print(f"Query file: {query_file}")
        print(f"Query emotion category: {query_emotion}")

        # Get recommendations
        recommendations = rec_system.find_similar(query_file,
n_recommendations=5)

        if recommendations:
            print("\nTop 5 similar tracks:")
            correct_predictions = 0
            total_predictions = len(recommendations)

            for i, (file_path, similarity) in enumerate(recommendations, 1):
                # Find the emotion of the recommended file
                rec_file_row = df[df['file_path'] == file_path]
                if len(rec_file_row) > 0:
                    rec_emotion = rec_file_row.iloc[0]['emotion']
                    if rec_emotion == query_emotion:
                        correct_predictions += 1
                    print(f"{i}. {os.path.basename(str(file_path))}
(similarity: {similarity:.3f}, emotion: {rec_emotion})")
                else:
                    print(f"{i}. {os.path.basename(str(file_path))}
(similarity: {similarity:.3f}, emotion: unknown)")

            # Calculate and print test accuracy
```

```
            test_accuracy = (correct_predictions / total_predictions) * 100 if
total_predictions > 0 else 0
            print(f"\nTest Accuracy: {test_accuracy:.1f}%
({correct_predictions}/{total_predictions} correct predictions)")
            print(f"Query emotion category: {query_emotion}")
        else:
            print("No recommendations found")

    print("\nWhoo Hoo! System completed successfully!")

if __name__ == "__main__":
    main()
```

Eda_analysis.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import os

class EDAAnalyzer:
    def __init__(self, output_dir="results"):
        self.output_dir = output_dir
        os.makedirs(output_dir, exist_ok=True)

        # Set a style for better plots
        plt.style.use('seaborn-v0_8')
        sns.set_palette("husl")

        # Training history storage
        self.training_history = {
            'discriminator_loss': [],
            'generator_loss': [],
            'epochs': []
        }

    def analyze_dataset(self, df, title="Music Emotion Dataset"):
        """Do EDA on the dataset"""
        print(f"\n{'='*60}")
        print(f"EXPLORATORY DATA ANALYSIS: {title}")
        print(f"{'='*60}")

        # Basic dataset information
        self._basic_info(df)

        print(f"\nEDA completed! Results saved in {self.output_dir}/")
```

```
    def _basic_info(self, df):
        """Basic dataset information"""
        print(f"\n1. BASIC DATASET INFORMATION")
        print(f"{'='*40}")
        print(f"Total samples: {len(df)}")
        print(f"Total features: {len(df.columns)}")
        print(f"Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f}
MB")
        print(f"Missing values: {df.isnull().sum().sum()}")
        print(f"Duplicate rows: {df.duplicated().sum()}")

        # Data types
        print(f"\nData types:")
        print(df.dtypes.value_counts())

    def compare_datasets(self, original_df, balanced_df):
        """Compare original and balanced datasets"""
        print(f"\n6. DATASET COMPARISON (Original vs Balanced)")
        print(f"{'='*50}")

        # Emotion distribution comparison
        original_dist = original_df['emotion'].value_counts()
        balanced_dist = balanced_df['emotion'].value_counts()

        comparison_df = pd.DataFrame({
            'Original': original_dist,
            'Balanced': balanced_dist
        }).fillna(0)

        print(f"\nEmotion distribution comparison:")
        print(comparison_df)

        # Create comparison plot
        plt.figure(figsize=(14, 6))

        # Before and after comparison
        plt.subplot(1, 2, 1)
        comparison_df.plot(kind='bar', ax=plt.gca())
        plt.title('Dataset Balance: Before vs After', fontsize=14,
fontweight='bold')
        plt.xlabel('Emotion Category', fontsize=12)
        plt.ylabel('Number of Samples', fontsize=12)
        plt.xticks(rotation=45)
        plt.legend(['Original', 'Balanced'])
        plt.grid(axis='y', alpha=0.3)

        # Balance ratio comparison
        plt.subplot(1, 2, 2)
        original_balance = original_dist.min() / original_dist.max()
```

```
        balanced_balance = balanced_dist.min() / balanced_dist.max()

        balance_data = pd.DataFrame({
            'Original': [original_balance],
            'Balanced': [balanced_balance]
        })

        balance_data.plot(kind='bar', ax=plt.gca(), color=['red', 'green'])
        plt.title('Balance Ratio Comparison', fontsize=14, fontweight='bold')
        plt.xlabel('Dataset', fontsize=12)
        plt.ylabel('Balance Ratio', fontsize=12)
        plt.xticks([0], ['Balance Ratio'])
        plt.legend(['Original', 'Balanced'])
        plt.grid(axis='y', alpha=0.3)

        plt.tight_layout()
        plt.savefig(f"{self.output_dir}/dataset_comparison.png", dpi=300,
bbox_inches='tight')
        plt.show()

        print(f"\nBalance improvement:")
        print(f"  Original balance ratio: {original_balance:.3f}")
        print(f"  Balanced balance ratio: {balanced_balance:.3f}")
        print(f"  Improvement: {((balanced_balance - original_balance) /
original_balance * 100):.1f}%")

    def record_training_step(self, epoch, d_loss, g_loss):
        """Record training step for visualization"""
        self.training_history['epochs'].append(epoch)
        self.training_history['discriminator_loss'].append(d_loss)
        self.training_history['generator_loss'].append(g_loss)

    def visualize_training(self):
        """Create comprehensive training visualizations"""
        print(f"\n7. MODEL TRAINING VISUALIZATION")
        print(f"{'='*50}")

        if not self.training_history['epochs']:
            print("No training history available for visualization")
            return

        # Create comprehensive training plots
        fig, axes = plt.subplots(2, 3, figsize=(18, 12))
        fig.suptitle('GAN Training Analysis', fontsize=16, fontweight='bold')

        # 1. Loss curves
        axes[0, 0].plot(self.training_history['epochs'],
self.training_history['discriminator_loss'],
                        'b-', label='Discriminator Loss', linewidth=2)
```

```
        axes[0, 0].plot(self.training_history['epochs'],
self.training_history['generator_loss'],
                        'r-', label='Generator Loss', linewidth=2)
        axes[0, 0].set_title('GAN Loss Curves', fontweight='bold')
        axes[0, 0].set_xlabel('Epoch')
        axes[0, 0].set_ylabel('Loss')
        axes[0, 0].legend()
        axes[0, 0].grid(True, alpha=0.3)

        # 2. Loss ratio analysis
        loss_ratio = [d/g if g > 0 else 0 for d, g in
zip(self.training_history['discriminator_loss'],
                                                                self.training_history
['generator_loss'])]
        axes[0, 1].plot(self.training_history['epochs'], loss_ratio, 'g-',
linewidth=2)
        axes[0, 1].axhline(y=1, color='k', linestyle='--', alpha=0.5,
label='Balanced (D/G=1)')
        axes[0, 1].set_title('Discriminator/Generator Loss Ratio',
fontweight='bold')
        axes[0, 1].set_xlabel('Epoch')
        axes[0, 1].set_ylabel('D/G Loss Ratio')
        axes[0, 1].legend()
        axes[0, 1].grid(True, alpha=0.3)

        # 3. Loss stability analysis
        d_loss_diff = np.diff(self.training_history['discriminator_loss'])
        g_loss_diff = np.diff(self.training_history['generator_loss'])
        axes[0, 2].plot(self.training_history['epochs'][1:], d_loss_diff, 'b-',
label='D Loss Change', alpha=0.7)
        axes[0, 2].plot(self.training_history['epochs'][1:], g_loss_diff, 'r-',
label='G Loss Change', alpha=0.7)
        axes[0, 2].axhline(y=0, color='k', linestyle='-', alpha=0.3)
        axes[0, 2].set_title('Loss Change Rate', fontweight='bold')
        axes[0, 2].set_xlabel('Epoch')
        axes[0, 2].set_ylabel('Loss Change')
        axes[0, 2].legend()
        axes[0, 2].grid(True, alpha=0.3)

        # 4. Training convergence analysis
        window_size = max(1, len(self.training_history['epochs']) // 10)
        d_loss_smooth =
np.convolve(self.training_history['discriminator_loss'],
                                    np.ones(window_size)/window_size,
mode='valid')
        g_loss_smooth = np.convolve(self.training_history['generator_loss'],
                                    np.ones(window_size)/window_size,
mode='valid')
```

```
        axes[1, 0].plot(self.training_history['epochs'][window_size-1:],
d_loss_smooth,
                        'b-', label='D Loss (Smoothed)', linewidth=2)
        axes[1, 0].plot(self.training_history['epochs'][window_size-1:],
g_loss_smooth,
                        'r-', label='G Loss (Smoothed)', linewidth=2)
        axes[1, 0].set_title('Smoothed Loss Curves', fontweight='bold')
        axes[1, 0].set_xlabel('Epoch')
        axes[1, 0].set_ylabel('Smoothed Loss')
        axes[1, 0].legend()
        axes[1, 0].grid(True, alpha=0.3)

        # 5. Training stability metrics
        d_loss_std = np.std(self.training_history['discriminator_loss'])
        g_loss_std = np.std(self.training_history['generator_loss'])
        d_loss_mean = np.mean(self.training_history['discriminator_loss'])
        g_loss_mean = np.mean(self.training_history['generator_loss'])

        stability_metrics = ['D Loss Mean', 'G Loss Mean', 'D Loss Std', 'G
Loss Std']
        stability_values = [d_loss_mean, g_loss_mean, d_loss_std, g_loss_std]

        bars = axes[1, 1].bar(stability_metrics, stability_values,
                              color=['skyblue', 'lightcoral', 'lightblue',
'lightpink'])
        axes[1, 1].set_title('Training Stability Metrics', fontweight='bold')
        axes[1, 1].set_ylabel('Value')
        axes[1, 1].tick_params(axis='x', rotation=45)

        # Add value labels on bars
        for bar, value in zip(bars, stability_values):
            axes[1, 1].text(bar.get_x() + bar.get_width()/2, bar.get_height() +
0.01,
                            f'{value:.3f}', ha='center', va='bottom',
fontsize=9)

        # 6. Training progress analysis
        progress_epochs = np.array(self.training_history['epochs'])
        progress_d_loss = np.array(self.training_history['discriminator_loss'])
        progress_g_loss = np.array(self.training_history['generator_loss'])

        # Calculate training phases
        total_epochs = len(progress_epochs)
        # early_phase = progress_epochs[:total_epochs//3]
        # mid_phase = progress_epochs[total_epochs//3:2*total_epochs//3]
        # late_phase = progress_epochs[2*total_epochs//3:]

        phases = ['Early', 'Mid', 'Late']
        d_loss_phases = [
```

```
            np.mean(progress_d_loss[:total_epochs//3]),
            np.mean(progress_d_loss[total_epochs//3:2*total_epochs//3]),
            np.mean(progress_d_loss[2*total_epochs//3:])
        ]
        g_loss_phases = [
            np.mean(progress_g_loss[:total_epochs//3]),
            np.mean(progress_g_loss[total_epochs//3:2*total_epochs//3]),
            np.mean(progress_g_loss[2*total_epochs//3:])
        ]

        x = np.arange(len(phases))
        width = 0.35

        axes[1, 2].bar(x - width/2, d_loss_phases, width,
label='Discriminator', color='skyblue')
        axes[1, 2].bar(x + width/2, g_loss_phases, width, label='Generator',
color='lightcoral')
        axes[1, 2].set_title('Training Phases Analysis', fontweight='bold')
        axes[1, 2].set_xlabel('Training Phase')
        axes[1, 2].set_ylabel('Average Loss')
        axes[1, 2].set_xticks(x)
        axes[1, 2].set_xticklabels(phases)
        axes[1, 2].legend()
        axes[1, 2].grid(True, alpha=0.3)

        plt.tight_layout()
        plt.savefig(f"{self.output_dir}/gan_training_analysis.png", dpi=300,
bbox_inches='tight')
        plt.close()

        # Print training analysis summary
        print(f"\nTraining Analysis Summary:")
        print(f"  Total epochs: {len(self.training_history['epochs'])}")
        print(f"  Final D loss: {self.training_history['discriminator_loss'][-
1]:.4f}")
        print(f"  Final G loss: {self.training_history['generator_loss'][-
1]:.4f}")
        print(f"  D loss range:
{min(self.training_history['discriminator_loss']):.4f} -
{max(self.training_history['discriminator_loss']):.4f}")
        print(f"  G loss range:
{min(self.training_history['generator_loss']):.4f} -
{max(self.training_history['generator_loss']):.4f}")
        print(f"  Training stability: D_std={d_loss_std:.4f},
G_std={g_loss_std:.4f}")

        # Determine training quality
        if d_loss_std < 0.1 and g_loss_std < 0.1:
            print(f"  Training quality: STABLE")
```

```
        elif d_loss_std < 0.2 and g_loss_std < 0.2:
            print(f"  Training quality: MODERATE")
        else:
            print(f"  Training quality: UNSTABLE")

        print(f"  Training visualization saved to:
{self.output_dir}/gan_training_analysis.png")
```

emotion_classifier.py

```
import pandas as pd
import numpy as np
class EmotionClassifier:
    def __init__(self):
        self.emotion_thresholds = {
            'happy': {
                'pitch_min': 60, 'pitch_max': 76,
                'velocity_min': 80, 'velocity_max': 112,
                'tempo_min': 120, 'tempo_max': 140,
                'note_overlap_ratio_min': 0.0, 'note_overlap_ratio_max': 0.4,
            },
            'sad': {
                'pitch_min': 48, 'pitch_max': 60,
                'velocity_min': 40, 'velocity_max': 70,
                'tempo_min': 60, 'tempo_max': 80,
                'note_overlap_ratio_min': 0.8, 'note_overlap_ratio_max': 1.0,
            },
            'excited': {
                'pitch_min': 65, 'pitch_max': 84,
                'velocity_min': 100, 'velocity_max': 127,
                'tempo_min': 110, 'tempo_max': 180,
                'note_overlap_ratio_min': 0.25, 'note_overlap_ratio_max': 0.65,
            },
            'hopeful': {
                'pitch_min': 60, 'pitch_max': 76,
                'velocity_min': 70, 'velocity_max': 100,
                'tempo_min': 100, 'tempo_max': 130,
                'note_overlap_ratio_min': 0.6, 'note_overlap_ratio_max': 0.8,
            },
            'tense': {
                'pitch_min': 50, 'pitch_max': 65,
                'velocity_min': 90, 'velocity_max': 127,
                'tempo_min': 100, 'tempo_max': 130,
                'note_overlap_ratio_min': 0.0, 'note_overlap_ratio_max': 0.3,
            },
            'fearful': {
                'pitch_min': 45, 'pitch_max': 60,
                'velocity_min': 50, 'velocity_max': 90,
```

```
                'tempo_min': 60, 'tempo_max': 100,
                'note_overlap_ratio_min': 0.0, 'note_overlap_ratio_max': 0.3,
            }
        }

    def classify_emotion(self, features):
        """Classify emotion based on feature thresholds (90% matching!)"""
        if not features:
            return 'unknown'

        emotion_scores = {}

        for emotion, thresholds in self.emotion_thresholds.items():
            score = 0
            total_criteria = 0

            # Check pitch range (use pitch_mean)
            if 'pitch_mean' in features:
                total_criteria += 1
                if thresholds['pitch_min'] <= features['pitch_mean'] <=
thresholds['pitch_max']:
                    score += 1

            # Check velocity range (use velocity_mean)
            if 'velocity_mean' in features:
                total_criteria += 1
                if thresholds['velocity_min'] <= features['velocity_mean'] <=
thresholds['velocity_max']:
                    score += 1

            # Check tempo range
            if 'tempo_mean' in features:
                total_criteria += 1
                if thresholds['tempo_min'] <= features['tempo_mean'] <=
thresholds['tempo_max']:
                    score += 1

            # Check note overlap ratio
            if 'note_overlap_ratio' in features:
                total_criteria += 1
                if thresholds['note_overlap_ratio_min'] <=
features['note_overlap_ratio'] <= thresholds['note_overlap_ratio_max']:
                    score += 1

            # Calculate percentage match
            if total_criteria > 0:
                emotion_scores[emotion] = score / total_criteria

        # Return emotion with highest score if >= 50% match (lowered from 75%)
```

```
        if emotion_scores:
            best_emotion = max(emotion_scores, key=emotion_scores.get)
            if emotion_scores[best_emotion] >= 0.5:
                return best_emotion

        return 'unknown'

    def classify_dataset(self, df):
        """Classify entire dataset"""
        emotions = []
        for _, row in df.iterrows():
            emotion = self.classify_emotion(row.to_dict())
            emotions.append(emotion)

        df['emotion'] = emotions
        return df

    def get_emotion_distribution(self, df):
        """Get distribution of emotions in dataset"""
        return df['emotion'].value_counts()
```

feature_extractor.py

```
# import os
import pandas as pd
import numpy as np
import pretty_midi
# from music21 import stream, pitch, key, meter, tempo

class MIDIFeatureExtractor:
    def __init__(self):
        self.features = []

    def extract_features(self, midi_path):
        """Extract all features from a MIDI file"""
        try:
            midi_data = pretty_midi.PrettyMIDI(midi_path)
            features = {}

            # Basic features
            features['file_path'] = midi_path
            features['duration'] = midi_data.get_end_time()

            # Pitch features
            features.update(self._extract_pitch_features(midi_data))

            # Velocity features
            features.update(self._extract_velocity_features(midi_data))
```

```
            # Tempo features
            features.update(self._extract_tempo_features(midi_data))

            # Harmony features
            features.update(self._extract_harmony_features(midi_data))

            # Rhythm features
            features.update(self._extract_rhythm_features(midi_data))

            # Articulation features
            features.update(self._extract_articulation_features(midi_data))

            # Temporal features
            features.update(self._extract_temporal_features(midi_data))

            # Dynamic features
            features.update(self._extract_dynamic_features(midi_data))

            return features

        except Exception as e:
            print(f"Error processing {midi_path}: {e}")
            return None

    def _extract_pitch_features(self, midi_data):
        """Extract pitch-related features"""
        all_notes = []
        for instrument in midi_data.instruments:
            for note in instrument.notes:
                all_notes.append(note.pitch)

        if not all_notes:
            return {'pitch_min': 0, 'pitch_max': 0, 'pitch_mean': 0,
'pitch_std': 0}

        return {
            'pitch_min': min(all_notes),
            'pitch_max': max(all_notes),
            'pitch_mean': np.mean(all_notes),
            'pitch_std': np.std(all_notes)
        }

    def _extract_velocity_features(self, midi_data):
        """Extract velocity-related features"""
        all_velocities = []
        for instrument in midi_data.instruments:
            for note in instrument.notes:
                all_velocities.append(note.velocity)
```

```
        if not all_velocities:
            return {'velocity_min': 0, 'velocity_max': 0, 'velocity_mean': 0,
'velocity_std': 0}

        return {
            'velocity_min': min(all_velocities),
            'velocity_max': max(all_velocities),
            'velocity_mean': np.mean(all_velocities),
            'velocity_std': np.std(all_velocities)
        }

    def _extract_tempo_features(self, midi_data):
        """Extract tempo-related features"""
        tempo_changes = midi_data.get_tempo_changes()
        if len(tempo_changes[1]) == 0:
            return {'tempo_mean': 120, 'tempo_std': 0}

        return {
            'tempo_mean': np.mean(tempo_changes[1]),
            'tempo_std': np.std(tempo_changes[1])
        }

    def _extract_harmony_features(self, midi_data):
        """Extract harmony-related features"""
        # Simple chord detection
        chord_changes = 0
        for instrument in midi_data.instruments:
            if instrument.is_drum:
                continue
            notes = sorted(instrument.notes, key=lambda x: x.start)
            for i in range(1, len(notes)):
                if abs(notes[i].start - notes[i-1].start) < 0.1:  #
Simultaneous notes
                    chord_changes += 1

        return {
            'chord_changes': chord_changes,
            'harmony_complexity': chord_changes / max(1,
len(midi_data.instruments))
        }

    def _extract_rhythm_features(self, midi_data):
        """Extract rhythm-related features"""
        all_durations = []
        for instrument in midi_data.instruments:
            for note in instrument.notes:
                all_durations.append(note.end - note.start)
```

```
        if not all_durations:
            return {'rhythm_mean': 0, 'rhythm_std': 0, 'syncopation': 0}

        # Simple syncopation measure
        syncopation = np.std(all_durations) / np.mean(all_durations) if
np.mean(all_durations) > 0 else 0

        return {
            'rhythm_mean': np.mean(all_durations),
            'rhythm_std': np.std(all_durations),
            'syncopation': syncopation
        }

    def _extract_articulation_features(self, midi_data):
        """Extract articulation features"""
        note_overlaps = 0
        total_notes = 0

        for instrument in midi_data.instruments:
            if instrument.is_drum:
                continue
            notes = sorted(instrument.notes, key=lambda x: x.start)
            total_notes += len(notes)
            for i in range(1, len(notes)):
                if notes[i].start < notes[i-1].end:  # Overlapping notes
                    note_overlaps += 1

        overlap_ratio = note_overlaps / max(1, total_notes)

        return {
            'note_overlap_ratio': overlap_ratio,
            'total_notes': total_notes
        }

    def _extract_temporal_features(self, midi_data):
        """Extract temporal features"""
        tempo_changes = midi_data.get_tempo_changes()
        tempo_variation = np.std(tempo_changes[1]) if len(tempo_changes[1]) > 1
else 0

        return {
            'tempo_variation': tempo_variation,
            'duration': midi_data.get_end_time()
        }

    def _extract_dynamic_features(self, midi_data):
        """Extract dynamic features"""
        all_velocities = []
        for instrument in midi_data.instruments:
```

```
            for note in instrument.notes:
                all_velocities.append(note.velocity)

        if not all_velocities:
            return {'dynamic_range': 0, 'dynamic_contrast': 0}

        dynamic_range = max(all_velocities) - min(all_velocities)
        dynamic_contrast = np.std(all_velocities)

        return {
            'dynamic_range': dynamic_range,
            'dynamic_contrast': dynamic_contrast
        }
```

recommendation_system.py

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import StandardScaler

class RecommendationSystem:
    def __init__(self):
        self.scaler = StandardScaler()
        self.feature_columns = None
        self.trained_data = None

    def prepare_features(self, df):
        """Prepare features for similarity calculation"""
        # Select numeric features only
        numeric_features =
df.select_dtypes(include=[np.number]).drop(['emotion'], axis=1,
errors='ignore')
        self.feature_columns = numeric_features.columns

        # Scale features
        scaled_features = self.scaler.fit_transform(numeric_features)

        return scaled_features

    def train(self, df):
        """Train recommendation system"""
        print("Training recommendation system...")

        # Prepare features
        features = self.prepare_features(df)

        # Store training data
```

```
        self.trained_data = df.copy()
        self.trained_data['scaled_features'] = list(features)

        print(f"Trained on {len(df)} samples")
        print(f"Feature dimensions: {features.shape[1]}")

    def find_similar(self, query_file_path, n_recommendations=5):
        """Find similar tracks to query file"""
        if self.trained_data is None:
            print("System not trained yet!")
            return []

        # Find query file in training data
        query_row = self.trained_data[self.trained_data['file_path'] ==
query_file_path]

        if len(query_row) == 0:
            print(f"Query file {query_file_path} not found in training data!")
            return []

        query_features = query_row.iloc[0]['scaled_features'].reshape(1, -1)
        query_emotion = query_row.iloc[0]['emotion']

        # Get all files with same emotion (ONLY from original dataset, not GAN-
generated)
        # Filter out GAN-generated samples by checking if file_path is not NaN
and exists
        same_emotion_data = self.trained_data[
            (self.trained_data['emotion'] == query_emotion) &
            (self.trained_data['file_path'].notna()) &
            (self.trained_data['file_path'] != '') &
            (self.trained_data['file_path'].str.contains('.mid', na=False))
        ]

        if len(same_emotion_data) <= 1:
            print(f"Not enough samples in emotion category: {query_emotion}")
            return []

        # Calculate similarities
        similarities = []
        for idx, row in same_emotion_data.iterrows():
            if row['file_path'] == query_file_path:
                continue  # Skip the query file itself

            # Skip if file_path is NaN or empty
            if pd.isna(row['file_path']) or row['file_path'] == '':
                continue

            features = row['scaled_features'].reshape(1, -1)
```

```
            similarity = cosine_similarity(query_features, features)[0][0]
            similarities.append((row['file_path'], similarity))

        # Sort by similarity and return top N
        similarities.sort(key=lambda x: x[1], reverse=True)

        return similarities[:n_recommendations]

    def get_emotion_stats(self):
        """Get statistics about trained data"""
        if self.trained_data is None:
            return {}

        return {
            'total_samples': len(self.trained_data),
            'emotion_distribution':
self.trained_data['emotion'].value_counts().to_dict(),
            'feature_dimensions': len(self.feature_columns) if
self.feature_columns is not None else 0
        }
```

simple_gan.py

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow import keras

class SimpleGAN:
    def __init__(self, feature_dim, noise_dim=100):
        self.feature_dim = feature_dim
        self.noise_dim = noise_dim
        self.scaler = StandardScaler()

        # Build generator and discriminator
        self.generator = self._build_generator()
        self.discriminator = self._build_discriminator()

        # Combined model for training generator
        self.combined = self._build_combined()

    def _build_generator(self):
        """simple generator"""
        model = keras.Sequential([
            keras.layers.Dense(256, activation='relu',
input_dim=self.noise_dim),
```

```
            keras.layers.Dense(512, activation='relu'),
            keras.layers.Dense(self.feature_dim, activation='tanh')
        ])
        return model

    def _build_discriminator(self):
        """simple discriminator"""
        model = keras.Sequential([
            keras.layers.Dense(512, activation='relu',
input_dim=self.feature_dim),
            keras.layers.Dense(256, activation='relu'),
            keras.layers.Dense(1, activation='sigmoid')
        ])
        return model

    def _build_combined(self):
        """combined model for training generator"""
        self.discriminator.trainable = False
        model = keras.Sequential([
            self.generator,
            self.discriminator
        ])
        return model

    def _compile_models(self):
        """Compile all models"""
        # discriminator
        self.discriminator.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5),
            loss='binary_crossentropy',
            metrics=['accuracy']
        )

        # combined model
        self.combined.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5),
            loss='binary_crossentropy'
        )

    def prepare_data(self, df, emotion):
        """Prepare data for specific emotion"""
        emotion_data = df[df['emotion'] == emotion]
        if len(emotion_data) == 0:
            return None, None

        # numeric features only
        numeric_features =
emotion_data.select_dtypes(include=[np.number]).drop(['emotion'], axis=1,
errors='ignore')
```

```
        # Scale features
        scaled_features = self.scaler.fit_transform(numeric_features)

        return scaled_features, numeric_features.columns

    def generate_samples(self, n_samples, emotion_data, feature_columns):
        """Generate new samples for specific emotion"""
        if emotion_data is None:
            return pd.DataFrame()

        # Generate noise
        noise = np.random.normal(0, 1, (n_samples, self.noise_dim))

        # Generate features
        generated_features = self.generator.predict(noise, verbose=0)

        # Inverse transform to original scale
        generated_features = self.scaler.inverse_transform(generated_features)

        # Create DataFrame
        generated_df = pd.DataFrame(generated_features,
columns=feature_columns)
        generated_df['emotion'] = emotion_data.iloc[0]['emotion'] if
len(emotion_data) > 0 else 'unknown'

        return generated_df

    def train(self, df, epochs=100, batch_size=32, eda_analyzer=None):
        """Train GAN on dataset"""
        # Compile models first
        self._compile_models()

        # Get unique emotions
        emotions = df['emotion'].unique()
        emotions = [e for e in emotions if e != 'unknown']

        # Prepare data for each emotion
        emotion_data = {}
        for emotion in emotions:
            data, columns = self.prepare_data(df, emotion)
            if data is not None:
                emotion_data[emotion] = (data, columns)

        # Training loop
        d_loss = 0
        g_loss = 0

        for epoch in range(epochs):
```

```
            for emotion in emotions:
                if emotion not in emotion_data:
                    continue

                data, columns = emotion_data[emotion]
                if len(data) < batch_size:
                    continue

                # Train discriminator
                real_data = data[np.random.choice(len(data), batch_size,
replace=True)]
                noise = np.random.normal(0, 1, (batch_size, self.noise_dim))
                fake_data = self.generator.predict(noise, verbose=0)

                # Combine real and fake data
                d_data = np.vstack([real_data, fake_data])
                d_labels = np.hstack([np.ones(batch_size),
np.zeros(batch_size)])

                # Train discriminator
                d_loss = self.discriminator.train_on_batch(d_data, d_labels)

                # Train generator
                noise = np.random.normal(0, 1, (batch_size, self.noise_dim))
                g_labels = np.ones(batch_size)
                g_loss = self.combined.train_on_batch(noise, g_labels)

            # Record training step for visualization
            if eda_analyzer:
                d_loss_val = d_loss[0] if isinstance(d_loss, (list,
np.ndarray)) else d_loss
                g_loss_val = g_loss[0] if isinstance(g_loss, (list,
np.ndarray)) else g_loss
                eda_analyzer.record_training_step(epoch, d_loss_val,
g_loss_val)

            if epoch % 20 == 0:
                d_loss_val = d_loss[0] if isinstance(d_loss, (list,
np.ndarray)) else d_loss
                g_loss_val = g_loss[0] if isinstance(g_loss, (list,
np.ndarray)) else g_loss
                print(f"Epoch {epoch}, D Loss: {d_loss_val:.4f}, G Loss:
{g_loss_val:.4f}")

    def balance_dataset(self, df, target_samples_per_emotion=200):
        """Balance dataset by generating samples for underrepresented
emotions"""
        emotion_counts = df['emotion'].value_counts()
        balanced_df = df.copy()
```

```
        for emotion, count in emotion_counts.items():
            if emotion == 'unknown':
                continue

            if count < target_samples_per_emotion:
                needed = target_samples_per_emotion - count
                print(f"Generating {needed} samples for {emotion}")

                # Get existing data for this emotion
                emotion_data = df[df['emotion'] == emotion]
                data, columns = self.prepare_data(df, emotion)

                if data is not None:
                    # Generate new samples
                    generated = self.generate_samples(needed, emotion_data,
columns)
                    balanced_df = pd.concat([balanced_df, generated],
ignore_index=True)

        return balanced_df
```

Fleiss-kappa.py

```
import pandas as pd
import numpy as np
from statsmodels.stats.inter_rater import fleiss_kappa

df = pd.read_csv("survey-results.csv")

emotion_cols = ["Happy", "Excited", "Tense", "Fearful", "Sad", "Hopeful"]

# p_ij - proportion of raters per emotion per track
n = 30
pij = df[emotion_cols] / n
print("\nSTEP 1: Proportions (p_ij):")
print(pij.round(3))

# P_i - agreement per track
pi = (df[emotion_cols] * (df[emotion_cols] - 1)).sum(axis=1) / (n * (n - 1))
print("\nSTEP 2: Agreement per track (P_i):")
print(pi.round(3))

# Mean observed agreement (P')
P_bar = pi.mean()
print(f"\nSTEP 3: Mean observed agreement (P'): {P_bar:.3f}")

# p_j - overall proportion per emotion and expected agreement (P'_e)
pj = df[emotion_cols].sum(axis=0) / (len(df) * n)
```

```python
P_e_bar = (pj ** 2).sum()
print("\nSTEP 4: Category proportions (p_j):")
print(pj.round(3))
print(f"Expected agreement by chance (P̄_e): {P_e_bar:.3f}")

# Fleiss' Kappa
kappa = (P_bar - P_e_bar) / (1 - P_e_bar)
print(f"\nSTEP 5: Fleiss' Kappa (κ): {kappa:.3f}")

# Interpretation (Landis & Koch, 1977)
if kappa < 0.00:
    interpretation = "Poor agreement"
elif kappa < 0.21:
    interpretation = "Slight agreement"
elif kappa < 0.41:
    interpretation = "Fair agreement"
elif kappa < 0.61:
    interpretation = "Moderate agreement"
elif kappa < 0.81:
    interpretation = "Substantial agreement"
else:
    interpretation = "Almost perfect agreement"

print(f"Interpretation: {interpretation}")
```

Appendix B – List of Figures

Fleiss Kappa Agreement System Results

Proportion of raters per category Per item (P_{ij})

```
STEP 1: Proportions (p_ij):
    Happy  Excited  Tense  Fearful    Sad  Hopeful
0   0.233    0.200  0.133    0.100  0.000    0.333
1   0.267    0.000  0.000    0.000  0.500    0.233
2   0.300    0.200  0.167    0.000  0.033    0.300
3   0.400    0.100  0.133    0.033  0.100    0.233
4   0.100    0.000  0.667    0.100  0.100    0.033
5   0.100    0.267  0.433    0.033  0.033    0.133
6   0.100    0.033  0.033    0.033  0.433    0.367
7   0.500    0.167  0.000    0.000  0.067    0.267
8   0.000    0.067  0.267    0.400  0.067    0.200
9   0.067    0.467  0.267    0.100  0.033    0.067
10  0.533    0.233  0.100    0.000  0.000    0.133
11  0.167    0.033  0.000    0.000  0.467    0.333
12  0.100    0.033  0.033    0.067  0.667    0.100
13  0.067    0.100  0.067    0.033  0.500    0.233
14  0.167    0.133  0.000    0.000  0.100    0.600
15  0.100    0.033  0.100    0.033  0.667    0.067
16  0.000    0.067  0.167    0.233  0.367    0.167
17  0.433    0.167  0.000    0.000  0.067    0.333
18  0.100    0.233  0.233    0.167  0.133    0.133
19  0.100    0.033  0.067    0.133  0.400    0.267
20  0.067    0.267  0.233    0.033  0.167    0.233
21  0.400    0.267  0.067    0.000  0.067    0.200
22  0.033    0.067  0.333    0.367  0.200    0.000
23  0.033    0.100  0.133    0.033  0.533    0.167
24  0.100    0.200  0.033    0.033  0.367    0.267
```

Per-track agreement scores (P_i) (0 = Track-1)

```
STEP 2: Agreement per track (P_i):
0     0.207
1     0.354
2     0.223
3     0.228
4     0.457
5     0.264
6     0.313
7     0.331
8     0.255
9     0.285
10    0.345
11    0.336
12    0.453
13    0.301
14    0.395
15    0.453
16    0.223
17    0.308
18    0.154
19    0.239
20    0.186
21    0.255
22    0.267
23    0.320
24    0.232
dtype: float64
```

Mean observed agreement ($\bar{P}$):

```
STEP 3: Mean observed agreement (P̄): 0.295
```

Expected chance agreement ($\bar{P}_e$): 0.184

```
Expected agreement by chance (P̄_e): 0.184
```

Overall Fleiss' Kappa (κ): 0.136

```
STEP 5: Fleiss' Kappa (κ): 0.136
Interpretation: Slight agreement
```

Recommendation System Results per track

Track-1-Q1__kJtgm1OUNA_0

```
Query file: ../EMOPIA_1.0/midis\Q1__kJtgm1OUNA_0.mid
Query emotion category: hopeful

Top 5 similar tracks:
1. Q3_FUAK5TBaNY8_2.mid (similarity: 0.891, emotion: hopeful)
2. Q3_wfXSdMsd4q8_1.mid (similarity: 0.859, emotion: hopeful)
3. Q3_wfXSdMsd4q8_4.mid (similarity: 0.833, emotion: hopeful)
4. Q4_ldCQ6N9G6Mk_0.mid (similarity: 0.801, emotion: hopeful)
5. Q3_FUAK5TBaNY8_3.mid (similarity: 0.796, emotion: hopeful)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: hopeful
```

Track-2-Q4_biROWEwkDQQ_1

```
Query file: ../EMOPIA_1.0/midis\Q4_biROWEwkDQQ_1.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q4_HYVmgq5Y93g_0.mid (similarity: 0.860, emotion: happy)
2. Q4_V3Y9L4UOcpk_1.mid (similarity: 0.841, emotion: happy)
3. Q3_02t1bj7ZABY_2.mid (similarity: 0.835, emotion: happy)
4. Q3_Ie5koh4qvJc_14.mid (similarity: 0.812, emotion: happy)
5. Q3_Ie5koh4qvJc_15.mid (similarity: 0.810, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-3-Q1_0vLPYiPN7qY_0

```
Query file: ../EMOPIA_1.0/midis\Q1_0vLPYiPN7qY_0.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q4_6kRPHamGDSo_2.mid (similarity: 0.884, emotion: happy)
2. Q1_KxlqB3j0zys_1.mid (similarity: 0.864, emotion: happy)
3. Q3_REq37pDAm3A_1.mid (similarity: 0.848, emotion: happy)
4. Q1_dfNdpy8TUzA_0.mid (similarity: 0.847, emotion: happy)
5. Q4_9eMxCs-LXCE_1.mid (similarity: 0.843, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-4-Q4_FyK_c-TIcCA_1

```
Query file: ../EMOPIA_1.0/midis\Q4_FyK_c-TIcCA_1.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q4_UpHutdJvZMI_2.mid (similarity: 0.850, emotion: happy)
2. Q3_-vAz_HTFEXs_1.mid (similarity: 0.847, emotion: happy)
3. Q4_Ie5koh4qvJc_27.mid (similarity: 0.838, emotion: happy)
4. Q3_c6CwY8Gbw0c_4.mid (similarity: 0.838, emotion: happy)
5. Q3_PLfFWFZflQU_1.mid (similarity: 0.835, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-5-Q4_ADJ_PDauy-g_1

```
Query file: ../EMOPIA_1.0/midis\Q4_ADJ_PDauy-g_1.mid
Query emotion category: sad

Top 5 similar tracks:
1. Q4_DvIH5yUrboc_2.mid (similarity: 0.922, emotion: sad)
2. Q4_I2MrA-o5H8I_1.mid (similarity: 0.901, emotion: sad)
3. Q4_UaYYMRkHMYc_2.mid (similarity: 0.880, emotion: sad)
4. Q4_heBmBQDWj-M_1.mid (similarity: 0.872, emotion: sad)
5. Q4_I2MrA-o5H8I_2.mid (similarity: 0.868, emotion: sad)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: sad
```

Track-6-Q2_jIKX3ShvLxo_2

```
Query file: ../EMOPIA_1.0/midis\Q2_jIKX3ShvLxo_2.mid
Query emotion category: sad

Top 5 similar tracks:
1. Q2_jIKX3ShvLxo_1.mid (similarity: 0.915, emotion: sad)
2. Q1_gSwv8hZGM-s_0.mid (similarity: 0.911, emotion: sad)
3. Q2_lalnGhxT3PQ_1.mid (similarity: 0.898, emotion: sad)
4. Q1_qlbazHayULg_8.mid (similarity: 0.890, emotion: sad)
5. Q2_ItGNJM6skM4_1.mid (similarity: 0.866, emotion: sad)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: sad
```

Track-7-Q4_B3aRl8iTEKw_1

```
Query file: ../EMOPIA_1.0/midis\Q4_B3aRl8iTEKw_1.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q4_egYSmNuIFGk_0.mid (similarity: 0.893, emotion: happy)
2. Q4_pRuwHN-lI44_0.mid (similarity: 0.880, emotion: happy)
3. Q4_Ie5koh4qvJc_32.mid (similarity: 0.865, emotion: happy)
4. Q3_fnqb8zqHqdY_0.mid (similarity: 0.863, emotion: happy)
5. Q4_FoTXpYZXxJs_0.mid (similarity: 0.858, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-8-Q1_6wFJhmhNeeg_0

```
Query file: ../EMOPIA_1.0/midis\Q1_6wFJhmhNeeg_0.mid
Query emotion category: tense

Top 5 similar tracks:
1. Q2_cm6E860vDjY_0.mid (similarity: 0.825, emotion: tense)
2. Q2_epX33OVpkmA_1.mid (similarity: 0.779, emotion: tense)
3. Q2_dtS02mrDMsM_2.mid (similarity: 0.719, emotion: tense)
4. Q2_UE0y8MHqT-g_2.mid (similarity: 0.689, emotion: tense)
5. Q2_JMt4m608s3k_2.mid (similarity: 0.534, emotion: tense)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: tense
```

Track-9-Q2_Dg935IbDggo_0

```
Query file: ../EMOPIA_1.0/midis\Q2_Dg935IbDggo_0.mid
Query emotion category: hopeful

Top 5 similar tracks:
1. Q1_jziI_cuN_H8_0.mid (similarity: 0.753, emotion: hopeful)
2. Q1_UOSlDydo94E_0.mid (similarity: 0.724, emotion: hopeful)
3. Q3_G4rL_OtfFAU_0.mid (similarity: 0.724, emotion: hopeful)
4. Q2_ANZf1QXsNrY_6.mid (similarity: 0.700, emotion: hopeful)
5. Q3_TYkTOwBfFB8_0.mid (similarity: 0.679, emotion: hopeful)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: hopeful
```

Track-10-Q2_9v2WSpn4FCw_4

```
Query file: ../EMOPIA_1.0/midis\Q2_9v2WSpn4FCw_4.mid
Query emotion category: hopeful

Top 5 similar tracks:
1. Q2_9v2WSpn4FCw_6.mid (similarity: 0.960, emotion: hopeful)
2. Q2_0v2N1ROvEI0_0.mid (similarity: 0.946, emotion: hopeful)
3. Q2_qlbazHayULg_0.mid (similarity: 0.920, emotion: hopeful)
4. Q3_uj3Gif77SYM_8.mid (similarity: 0.870, emotion: hopeful)
5. Q2_k-FNDbK6Qhg_0.mid (similarity: 0.843, emotion: hopeful)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: hopeful
```

Track-11-Q1__8v0MFBZoco_1

```
Query file: ../EMOPIA_1.0/midis\Q1__8v0MFBZoco_1.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q2_Q5b5unyP8BM_0.mid (similarity: 0.906, emotion: happy)
2. Q2_7n32BtYb4d4_2.mid (similarity: 0.904, emotion: happy)
3. Q2_7n32BtYb4d4_0.mid (similarity: 0.896, emotion: happy)
4. Q1__8v0MFBZoco_0.mid (similarity: 0.896, emotion: happy)
5. Q1_e838BE_MH6s_4.mid (similarity: 0.894, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-12-Q4_7yW9c7t8Hq0_0

```
Query file: ../EMOPIA_1.0/midis\Q4_7yW9c7t8Hq0_0.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q4_vBHoSz8cJIE_4.mid (similarity: 0.928, emotion: happy)
2. Q4_Tc684nD8CEA_1.mid (similarity: 0.927, emotion: happy)
3. Q3_MbvPxCUMSek_2.mid (similarity: 0.917, emotion: happy)
4. Q3_QJlnTN7HRwE_1.mid (similarity: 0.904, emotion: happy)
5. Q4_NP0lwB-_-og_0.mid (similarity: 0.903, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-13-Q2_08t73Qgjkt4_0

```
Query file: ../EMOPIA_1.0/midis\Q2_08t73Qgjkt4_0.mid
Query emotion category: sad

Top 5 similar tracks:
1. Q4_1vjy9oMFa8c_0.mid (similarity: 0.870, emotion: sad)
2. Q3_2v6gJi03LlA_0.mid (similarity: 0.850, emotion: sad)
3. Q2_1kny88W533Q_0.mid (similarity: 0.849, emotion: sad)
4. Q3_E5qEloUO3SM_1.mid (similarity: 0.842, emotion: sad)
5. Q3_PXOWy7NiZhk_0.mid (similarity: 0.834, emotion: sad)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: sad
```

Track-14-Q3_veG92Oi-DlU_0

```
Query file: ../EMOPIA_1.0/midis\Q3_veG92Oi-DlU_0.mid
Query emotion category: sad

Top 5 similar tracks:
1. Q3_g-yM0Lsp4lc_0.mid (similarity: 0.965, emotion: sad)
2. Q4_mGYKWCVnMwQ_0.mid (similarity: 0.940, emotion: sad)
3. Q4_xgtwQGeB6_0_0.mid (similarity: 0.923, emotion: sad)
4. Q3_TonQX8XbvX8_2.mid (similarity: 0.892, emotion: sad)
5. Q4_mGYKWCVnMwQ_1.mid (similarity: 0.889, emotion: sad)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: sad
```

Track-15-Q1_5Ju9q1N2x0E_2

```
Query file: ../EMOPIA_1.0/midis\Q1_5Ju9q1N2x0E_2.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q1_Ie5koh4qvJc_6.mid (similarity: 0.852, emotion: happy)
2. Q4_ldCQ6N9G6Mk_5.mid (similarity: 0.812, emotion: happy)
3. Q1_yFw_kO7DF-Y_2.mid (similarity: 0.768, emotion: happy)
4. Q3_bbU31JLtlug_1.mid (similarity: 0.765, emotion: happy)
5. Q2_CxFXrYgdSSI_2.mid (similarity: 0.758, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-16-Q3_vVTP0DOL_2Q_0

```
Query file: ../EMOPIA_1.0/midis\Q3_vVTP0DOL_2Q_0.mid
Query emotion category: sad

Top 5 similar tracks:
1. Q3_vVTP0DOL_2Q_3.mid (similarity: 0.985, emotion: sad)
2. Q3_vVTP0DOL_2Q_1.mid (similarity: 0.969, emotion: sad)
3. Q3_wfXSdMsd4q8_0.mid (similarity: 0.951, emotion: sad)
4. Q3_K84TcgjCRt4_0.mid (similarity: 0.920, emotion: sad)
5. Q3_gvWDOIiocuE_2.mid (similarity: 0.899, emotion: sad)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: sad
```

Track-17-Q3_wlOa1jkTn_U_2

```
Query file: ../EMOPIA_1.0/midis\Q3_wlOa1jkTn_U_2.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q2_S1T3UF1vhSk_1.mid (similarity: 0.906, emotion: happy)
2. Q1_60LLKmpgzRM_0.mid (similarity: 0.894, emotion: happy)
3. Q3_ilCcZpD39HU_2.mid (similarity: 0.894, emotion: happy)
4. Q4_NP0lwB-_-og_0.mid (similarity: 0.887, emotion: happy)
5. Q4_d_49EtXDMFE_2.mid (similarity: 0.878, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-18-Q1__SJQaaRzD-A_1

```
Query file: ../EMOPIA_1.0/midis\Q1__SJQaaRzD-A_1.mid
Query emotion category: hopeful

Top 5 similar tracks:
1. Q1_pNwQ9Tu_bCs_2.mid (similarity: 0.886, emotion: hopeful)
2. Q1_pNwQ9Tu_bCs_0.mid (similarity: 0.859, emotion: hopeful)
3. Q1_zmZHNy9T8Pg_2.mid (similarity: 0.847, emotion: hopeful)
4. Q2_XC_SiJszQx0_1.mid (similarity: 0.840, emotion: hopeful)
5. Q1_8rupdevqfuI_0.mid (similarity: 0.832, emotion: hopeful)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: hopeful
```

Track-19-Q2_b8HVQtIoBYU_1

```
Query file: ../EMOPIA_1.0/midis\Q2_b8HVQtIoBYU_1.mid
Query emotion category: hopeful

Top 5 similar tracks:
1. Q2_P-We4q3goKU_5.mid (similarity: 0.785, emotion: hopeful)
2. Q2_x5ibvz38jOs_1.mid (similarity: 0.782, emotion: hopeful)
3. Q2_cQp1BYDGcRo_0.mid (similarity: 0.674, emotion: hopeful)
4. Q1_dfNdpy8TUzA_1.mid (similarity: 0.649, emotion: hopeful)
5. Q1_8rupdevqfuI_0.mid (similarity: 0.629, emotion: hopeful)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: hopeful
```

Track-20-Q4_9v2WSpn4FCw_9

```
Query file: ../EMOPIA_1.0/midis\Q4_9v2WSpn4FCw_9.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q1_T92R7xjce34_1.mid (similarity: 0.924, emotion: happy)
2. Q3_ltxNPJda7zE_2.mid (similarity: 0.846, emotion: happy)
3. Q1_9v2WSpn4FCw_5.mid (similarity: 0.840, emotion: happy)
4. Q1_9v2WSpn4FCw_0.mid (similarity: 0.789, emotion: happy)
5. Q2_epX33OVpkmA_2.mid (similarity: 0.739, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-21-Q2_ANZf1QXsNrY_7

```
Query file: ../EMOPIA_1.0/midis\Q2_ANZf1QXsNrY_7.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q1_NGE9ynTJABg_0.mid (similarity: 0.852, emotion: happy)
2. Q2_cm6E860vDjY_1.mid (similarity: 0.844, emotion: happy)
3. Q1_7yW9c7t8Hq0_1.mid (similarity: 0.834, emotion: happy)
4. Q1_jziI_cuN_H8_2.mid (similarity: 0.831, emotion: happy)
5. Q2_qlbazHayULg_6.mid (similarity: 0.813, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-22-Q1_2Z9SjI131jA_0

```
Query file: ../EMOPIA_1.0/midis\Q1_2Z9SjI131jA_0.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q4_gxsdWKc1QaM_0.mid (similarity: 0.880, emotion: happy)
2. Q4_iHPKusssXzk_0.mid (similarity: 0.870, emotion: happy)
3. Q1_2Z9SjI131jA_12.mid (similarity: 0.868, emotion: happy)
4. Q2_5CEAeMiXKaA_0.mid (similarity: 0.865, emotion: happy)
5. Q4_k6aLLVX1D7Y_1.mid (similarity: 0.860, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-23-Q3_7JIdJLkJ0S4_1

```
Query file: ../EMOPIA_1.0/midis\Q3_7JIdJLkJ0S4_1.mid
Query emotion category: sad

Top 5 similar tracks:
1. Q1_im4Qxn3GQvo_0.mid (similarity: 0.858, emotion: sad)
2. Q2_Ie5koh4qvJc_10.mid (similarity: 0.842, emotion: sad)
3. Q3_wlOa1jkTn_U_0.mid (similarity: 0.809, emotion: sad)
4. Q1_GbUV3TXUzeQ_2.mid (similarity: 0.801, emotion: sad)
5. Q1_1Qc15G0ZHIg_1.mid (similarity: 0.797, emotion: sad)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: sad
```

Track-24-Q3_Adil-zpJdEs_0

```
Query file: ../EMOPIA_1.0/midis\Q3_Adil-zpJdEs_0.mid
Query emotion category: happy

Top 5 similar tracks:
1. Q3__vZOEQCYSaY_2.mid (similarity: 0.904, emotion: happy)
2. Q3_K84TcgjCRt4_2.mid (similarity: 0.892, emotion: happy)
3. Q4_HYVmgq5Y93g_0.mid (similarity: 0.890, emotion: happy)
4. Q3_2v6gJi03LlA_1.mid (similarity: 0.874, emotion: happy)
5. Q3_TYkTOwBfFB8_1.mid (similarity: 0.869, emotion: happy)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: happy
```

Track-25-Q3_2eIsQtm4YNs_1

```
Query file: ../EMOPIA_1.0/midis\Q3_2eIsQtm4YNs_1.mid
Query emotion category: sad

Top 5 similar tracks:
1. Q4_bfopzItCYrE_2.mid (similarity: 0.858, emotion: sad)
2. Q1_K6OFDxBU370_0.mid (similarity: 0.825, emotion: sad)
3. Q4_fOYX0uH8mSQ_0.mid (similarity: 0.821, emotion: sad)
4. Q4_a5QCcwEjxAk_0.mid (similarity: 0.820, emotion: sad)
5. Q4_JxSU49jFKwM_1.mid (similarity: 0.803, emotion: sad)

Test Accuracy: 100.0% (5/5 correct predictions)
Query emotion category: sad
```

Appendix C – Ethical Approval

University of Chester

Faculty of Science, Business & Enterprise
Science & Engineering Research Ethics Committee

Kethmie Perera
via online system

11th June 2025

Dear Kethmie,

Study title: ENHANCING MUSIC EMOTION RECOGNITION FOR EMOTION BASED MUSIC RECOMMENDATION SYSTEM USING GENERATIVE ADVERSARIAL NETWORKS (GAN)
FSE-REC reference: 373/25/CES/KP
Version number: 2

Thank you for responding to my letter dated 16/05/2025 and resubmitting your application for ethical approval.

Decision: Approved

I am satisfied that you have considered any potential ethical issues and taken steps to mitigate against these, and am therefore pleased to confirm ethical approval, provided that you comply with the conditions set out in the University's Code of Practice for Research and adhere to the processes described in your application form and supporting documentation.

Yours sincerely,

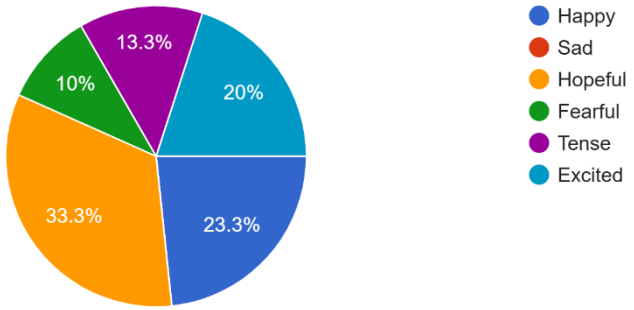
Assoc. Prof. Simon Oliver
Chair, Science and Engineering Research Ethics Committee

cc. Stuart Cunningham (Supervisor)

Appendix D – Survey Results

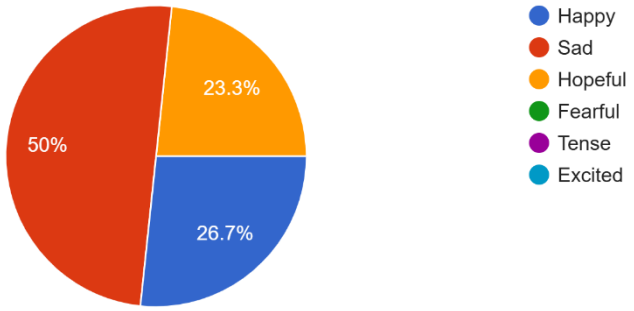
What was the main emotion you felt while listening to Track 1? (Select one) - Press to play track 1

30 responses



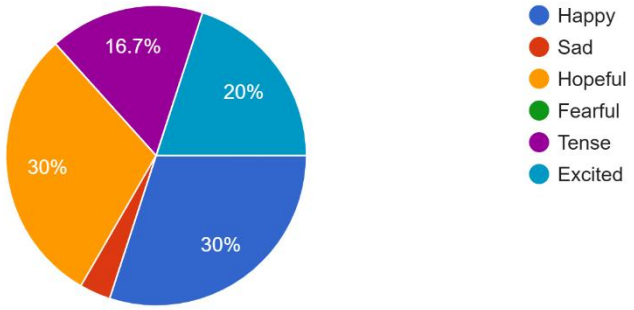
What was the main emotion you felt while listening to Track 2? (Select one) - Press to play track 2

30 responses



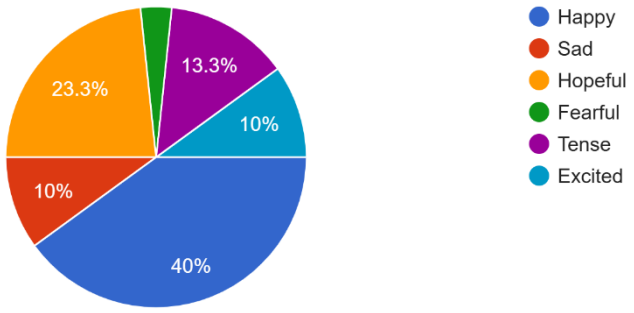
What was the main emotion you felt while listening to Track 3? (Select one) - Press to play track 3

30 responses



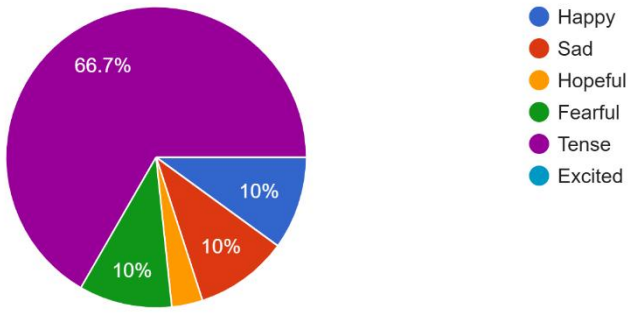
What was the main emotion you felt while listening to Track 4? (Select one) - Press to play track 4

30 responses



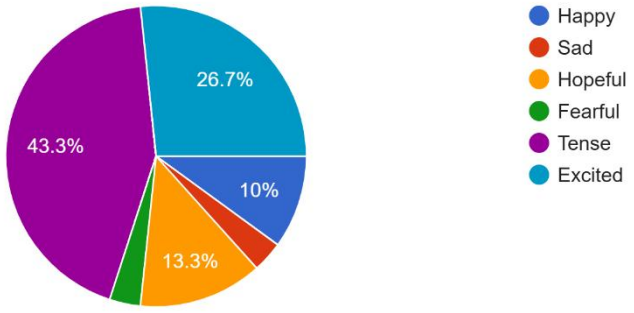
What was the main emotion you felt while listening to Track 5? (Select one) - Press to play track 5

30 responses



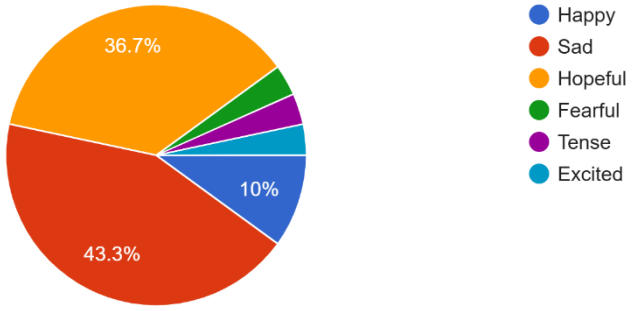
What was the main emotion you felt while listening to Track 6? (Select one) - Press to play track 6

30 responses



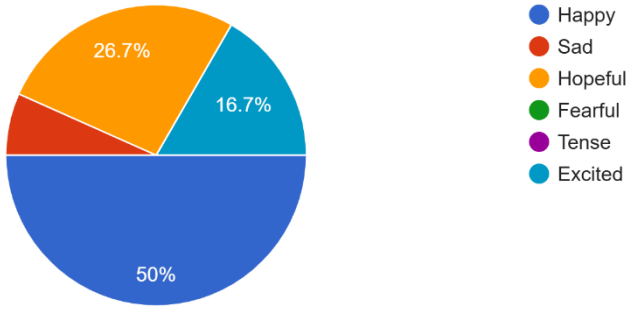
What was the main emotion you felt while listening to Track 7? (Select one) - Press to play track 7

30 responses



What was the main emotion you felt while listening to Track 8? (Select one) - Press to play track 8

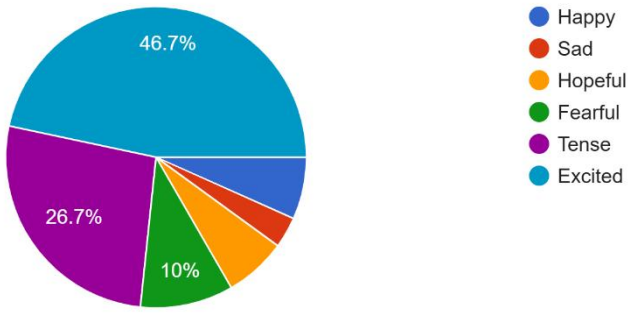
30 responses



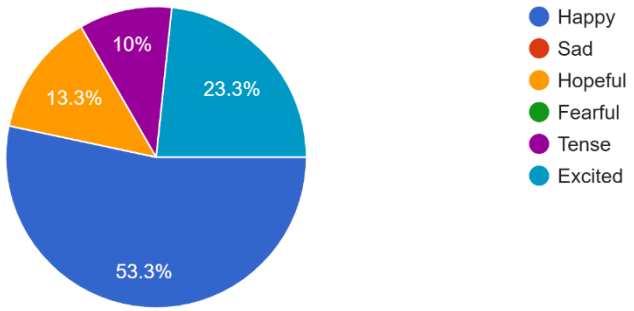
What was the main emotion you felt while listening to Track 9? (Select one) - Press to play track 9

30 responses

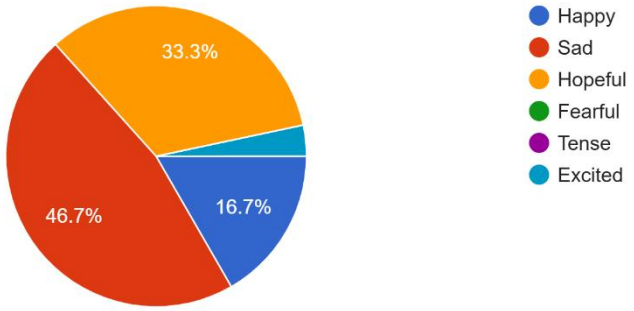
What was the main emotion you felt while listening to Track 10? (Select one) - Press to play track 10
30 responses



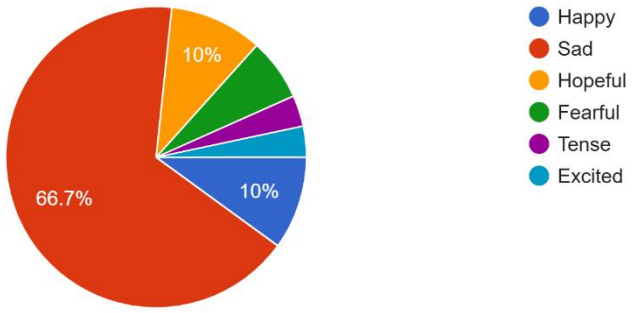
What was the main emotion you felt while listening to Track 11? (Select one) - Press to play track 11
30 responses



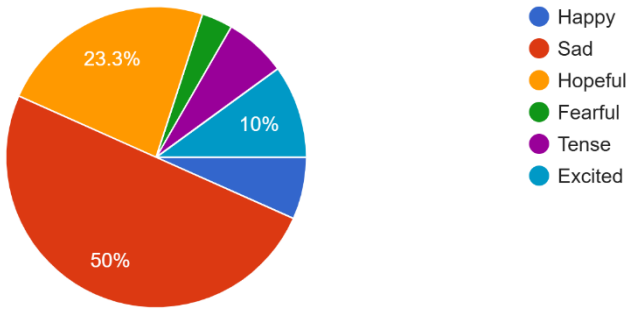
What was the main emotion you felt while listening to Track 12? (Select one) - Press to play track 12
30 responses



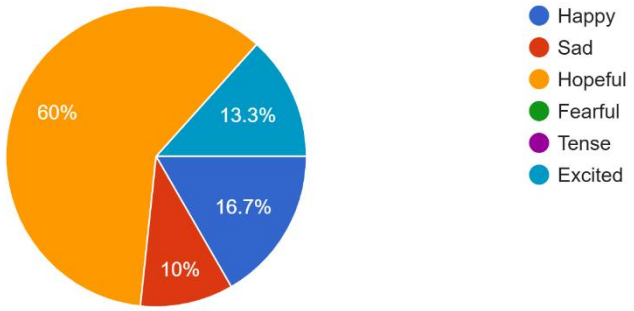
What was the main emotion you felt while listening to Track 13? (Select one) - Press to play track 13
30 responses



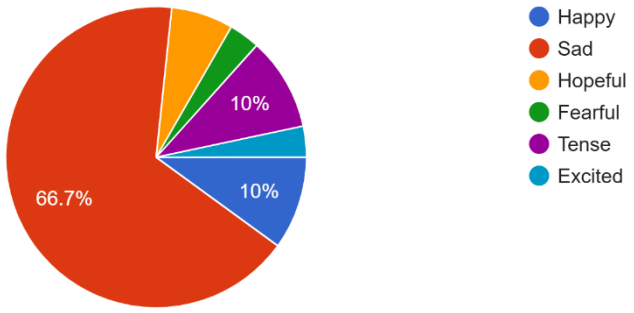
What was the main emotion you felt while listening to Track 14? (Select one) - Press to play track 14
30 responses



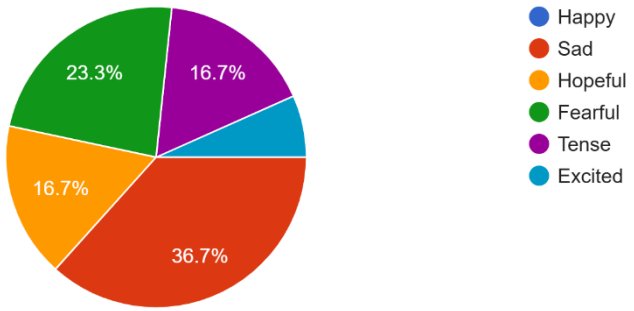
What was the main emotion you felt while listening to Track 15? (Select one) - Press to play track 15
30 responses



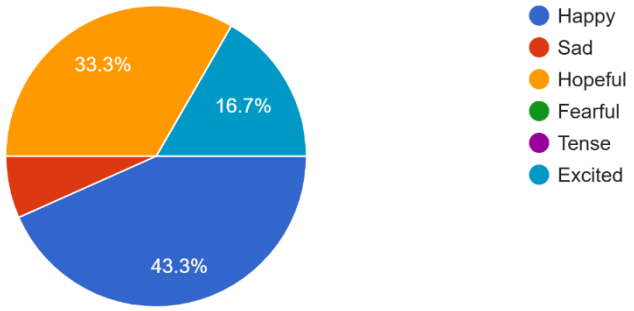
What was the main emotion you felt while listening to Track 16? (Select one) - Press to play track 16
30 responses



What was the main emotion you felt while listening to Track 17? (Select one) - Press to play track 17
30 responses

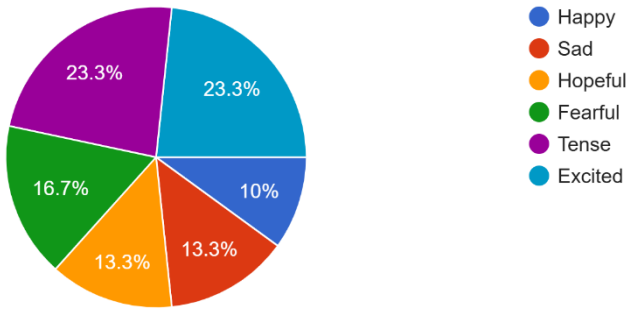


What was the main emotion you felt while listening to Track 18? (Select one) - Press to play track 18
30 responses



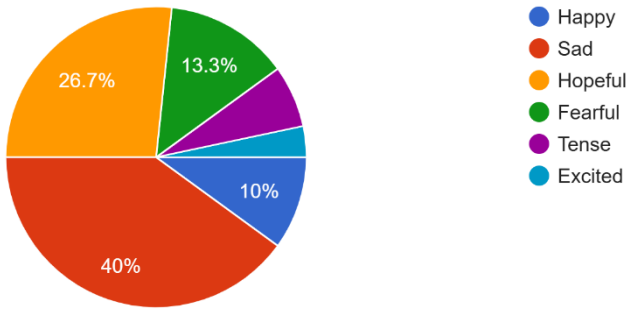
What was the main emotion you felt while listening to Track 19? (Select one) - Press to play track 19

30 responses



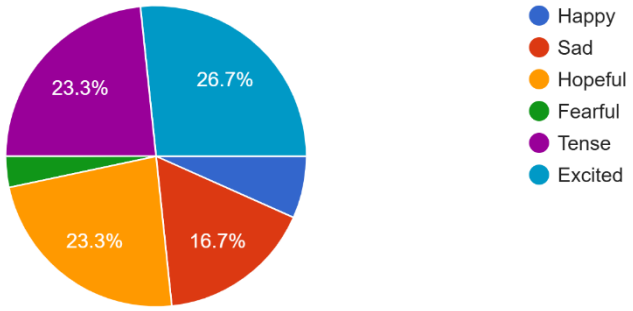
What was the main emotion you felt while listening to Track 20? (Select one) - Press to play track 20

30 responses

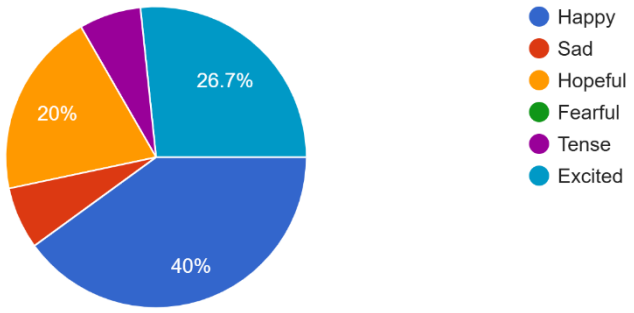


What was the main emotion you felt while listening to Track 21? (Select one) - Press to play track 21

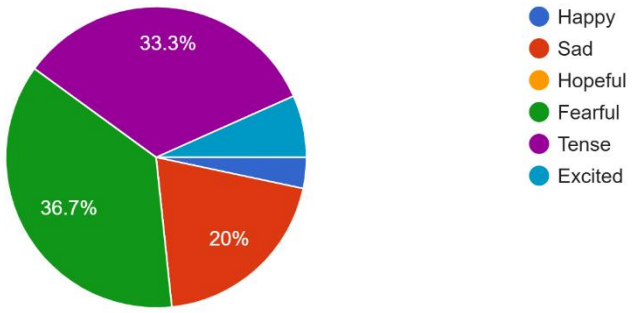
30 responses



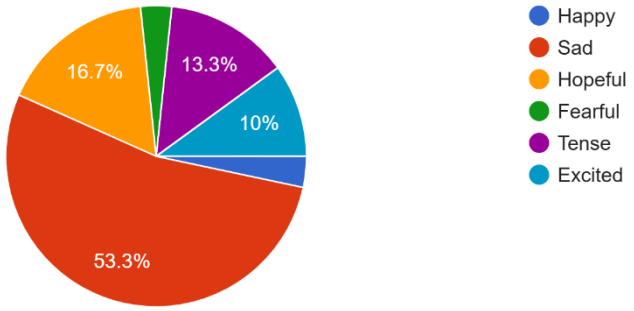
What was the main emotion you felt while listening to Track 22? (Select one) - Press to play track 22
30 responses



What was the main emotion you felt while listening to Track 23? (Select one) - Press to play track 23
30 responses

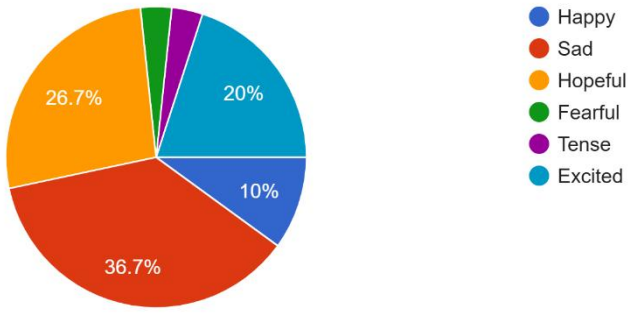


What was the main emotion you felt while listening to Track 24? (Select one) - Press to play track 24
30 responses



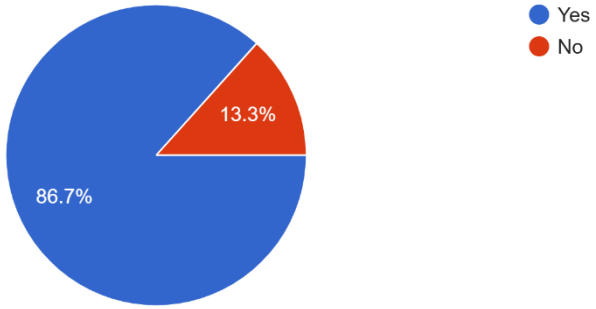
What was the main emotion you felt while listening to Track 25? (Select one) - Press to play track 25

30 responses



Did you find the task easy to complete?

30 responses



Do you have any feedback or suggestions regarding this session?

11 responses

Maybe add an option called Neutral too?

It would be better if you could add some Fearful emotion related tracks more. Overall experience is great.

It's good

No worked perfectly

fearful wasn't an emotion that I could clearly perceive from listening to the tunes so I didn't record feeling that a lot

Very good survey! Interesting idea.

I would like , not only melodies But I think voices and words are also good to recognise emotions... Cause melody,words,voices express emotions...right? I guess it would nice if the above i mentioned things added. 😊
But overall it's a very good session and I enjoyed filling it. 😊